भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

ADVANCED 5G/ 6G WIRELESS COMMUNICATIONS

DATA HANDLING AND HARDWARE PROGRAMMING

AT

FUTURE WIRELESS COMMUNICATIONS,

INDIAN INSTITUTE OF TECHNOLOGY – HYDERABAD

Internship Report

Submitted in Partial Fulfilment of the Requirement for

Awarding the Degree

Of

BACHELOR OF TECHNOLOGY

In

AERONAUTICAL ENGINEERING

By

Tirumala Sai Nithin
Roll Number: 20N31A2135
Employee ID: FWC22187

Under the esteemed guidance of,

Dr. G. V. V Sharma

Associate Professor
Electrical Engineering
Indian Institute of Technology Hyderabad
Kandi, Sangareddy-502285, Telangana, India.

भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

CERTIFICATE OF COMPLETION

This is to certify that **Tirumala Sai Nithin**, a student of **Malla Reddy College of Engineering and Technology**, has successfully completed an internship at **Future Wireless Communications (FWC), Indian Institute of Technology (IIT) Hyderabad**.

The internship was undertaken in the area of **advanced 5G/6G wireless communication**, with a focus on **Data Handling and Hardware Programming**. Throughout the internship period, he demonstrated commendable dedication, diligence, and technical proficiency in carrying out assigned tasks and projects.

This certificate is awarded in recognition of **Tirumala Sai Nithin's** valuable contributions, innovative ideas, and commitment to excellence during their tenure at FWC, IIT Hyderabad. We believe that the skills and knowledge acquired during this internship will serve as a solid foundation for their future endeavors in the field of wireless communications and beyond.

Dr. G. V. V. Sharma
Associate Professor
Department of Electrical Engineering
Future Wireless Communications (FWC)
Indian Institute of Technology (IIT) Hyderabad

ACKNOWLEDGEMENT

I would like to extend my sincere appreciation to all those who have played a crucial role in the successful completion of my internship report at the Future Wireless Communication (FWC), IIT Hyderabad.

Firstly, I am deeply grateful to **Dr. G. V. V. Sharma**, my internship head, for their unwavering guidance, support, and encouragement throughout this enriching experience. Their expertise and mentorship have significantly contributed to my professional development in **Data Handling and Hardware Programming**.

I would also like to express my gratitude to the entire team at FWC for providing me with an inspiring and collaborative environment to learn and thrive. Their cooperation and assistance have been indispensable in the accomplishment of my research objectives.

Furthermore, I extend my heartfelt thanks to the HR department of the company for their assistance in facilitating my internship placement and ensuring a smooth transition into the organization.

I am also indebted to the principal, faculty members, and staff of **Malla Reddy College of Engineering and Technology** for providing me with the opportunity to pursue this internship. Their support and encouragement have been instrumental in broadening my horizons and gaining practical insights into the field of [mention your field of study].

Last but not least, I am immensely grateful to my family and friends for their unwavering support and encouragement throughout this journey.

Thank you all for your invaluable contributions and belief in my abilities.

By,
Tirumala Sai Nithin,
FWC22187.

DECLARATION

I, **Tirumala Sai Nithin**, hereby declare that the internship report entitled **"Data Handling and Hardware Programming"** is an authentic representation of my work carried out during my internship at Future Wireless Communications (FWC), IIT Hyderabad, in the field of advanced 5G/6G wireless communication under the domain of Data Handling and Hardware Programming.

I affirm that the content presented in this report is the result of my independent research and analysis conducted under the supervision of **Dr. G. V. V. Sharma**. Wherever external sources have been utilized, proper citations and references have been provided to acknowledge the original authors and sources.

I further declare that this report has not been submitted, in part or in full, for the fulfillment of any other degree or qualification at any institution. Any similarities found with the work of others are purely coincidental and unintentional.

I take full responsibility for the accuracy, authenticity, and originality of the content presented in this report. Any opinions expressed herein are solely my own and do not necessarily reflect the views of Future Wireless Communications (FWC), IIT Hyderabad, or any other affiliated organization.

I understand the consequences of academic dishonesty and plagiarism and affirm that all ethical standards and guidelines have been adhered to throughout the preparation of this report.

Place: Hyderabad

By,

Tirumala Sai Nithin,

FWC22187.

భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Certificate of Completion

This is to certify that

Mr. Tirumala Sai Nithin

(FWC- 222187)

has successfully completed the certificate course

Module-1 (Data Handling & Hardware Programming) in Future Wireless Communication program

(December 2023 - February 2024)

Prof. Kiran Kumar Kuchi
Project Leader
IIT Hyderabad

CERTIFICATE OF COMPLETION
IIT HYDERABAD

Prof. A. Rajagopal
Chair, Centre for Continued Education(CCE)
IIT Hyderabad

ఐఐటీ హైదరాబాద్
आई आई टी हैदराबाद
IIT Hyderabad

Indian Institute of Technology
Hyderabad National Highway 9
Kandi, Sangareddy
Telangana - 502284
Phone: (040) 2301 6772.
Fax: (040) 2301 6000

Future Wireless Communication 5G Advanced/6G Project
INTERNSHIP COMPLETION CERTIFICATE

Date :29 March 2024

This is to certify that **Mr. Tirumala Sai Nithin** , has carried out "**Future Wireless Communications**" **Internship** at **Indian Institute of Technology Hyderabad**, under our supervision during the period from **1st-December-2023 to 29th-March-2024**. His role and responsibilities include **"Data Handling and Hardware programming , INTERCHIP** using **VAMAN**" under **Prof. G. V. V. Sharma.**

He is extremely sincere and hardworking.

We wish you every success in all his future endeavors.

Prof. GVV Sharma
Program Coordinator
IIT Hyderabad

TABLE OF CONTENTS

CHAPTER-1

INTRODUCTION

1.1 TERMUX

Termux is an Android terminal emulator and Linux environment application that allows users to run various Linux packages and commands on their Android devices. It's a powerful tool that enables users to utilize a full-fledged Linux distribution directly on their smartphones or tablets. Termux provides a command-line interface (CLI) environment similar to those found on desktop or server Linux distributions.

Here's a breakdown of its key features and functionalities:

1.1.1. Terminal Emulator: Termux provides a terminal emulator that simulates a Linux shell environment directly on Android devices. Users interact with this environment through a command-line interface, typing commands and receiving text-based outputs.

1.1.2. Package Management: Termux offers a package management system similar to those found in mainstream Linux distributions. Users can install, update, and remove packages using the package manager, which is a fork of the Debian APT (Advanced Package Tool). This allows users to easily install various software packages, including programming languages, development tools, utilities, and more.

1.1.3. Wide Range of Packages: Termux provides access to a vast repository of precompiled packages, including popular programming languages like Python, Ruby, Perl, and Node.js, as well as development tools like Git, Vim, Emacs, and compilers. Additionally, various networking utilities, system administration tools, and penetration testing tools are available.

1.1.4. Customization: Users can customize their Termux environment to suit their needs. This includes configuring the shell environment, installing additional packages, setting up SSH and VNC servers, and even running a full-fledged Linux desktop environment like XFCE or LXDE.

1.1.5. Scripting and Automation: With Termux, users can write and execute shell scripts directly on their Android devices. This enables automation of various tasks and workflows, making it a powerful tool for developers, system administrators, and power users.

1.1.6. Access to Device APIs: Termux provides access to various device APIs and sensors, allowing users to interact with hardware components such as the camera, microphone,

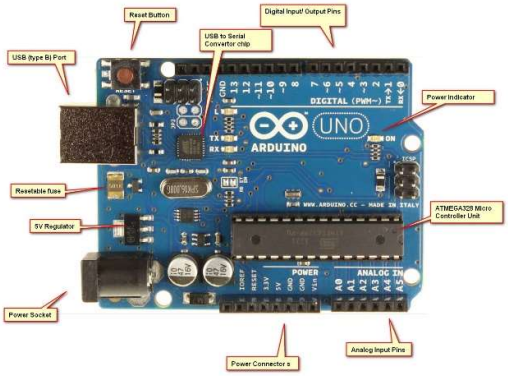
accelerometer, and GPS. This enables the development of Android applications and scripts that leverage these capabilities.

1.1.7. Community Support: Termux has a vibrant and active community of users and developers who contribute packages, share scripts and workflows, and provide support and assistance through forums, chat channels, and online resources.

Overall, Termux is a versatile and powerful tool that brings the flexibility and capabilities of a Linux environment to Android devices, empowering users to perform a wide range of tasks and activities directly from their smartphones or tablets. Whether you're a developer, sysadmin, or enthusiast, Termux provides a convenient and accessible platform for running Linux software and scripts on the go.

1.2 Arduino Board

Arduino is an open-source electronics platform based on easy-to-use hardware and software. It consists of a programmable circuit board (often referred to as an Arduino board) and an integrated development environment (IDE) for writing and uploading code to the board. Arduino boards are designed to be accessible to beginners and professionals alike, enabling users to create interactive projects and prototypes without needing extensive electronics knowledge.



Here's a detailed explanation of the components and features of an Arduino board:

1.2.1. Microcontroller: At the heart of every Arduino board is a microcontroller. The microcontroller is a small computer on a single integrated circuit containing a processor core, memory, and programmable input/output peripherals. The most commonly used microcontroller in Arduino boards is the Atmel AVR series, particularly the ATmega328P, although newer boards might use different microcontrollers like the ARM-based SAMD21 or ESP8266/ESP32 for Wi-Fi connectivity.

1.2.2. Input/Output (I/O) Pin: Arduino boards feature a variety of digital and analog input/output pins, which allow users to connect sensors, actuators, displays, and other

electronic components to the board. These pins can be configured as either digital inputs or outputs, or as analog inputs for reading analog sensor values.

1.2.3. Power Supply: Arduino boards can be powered via USB connection, a DC power jack, or by using an external power source such as batteries. The boards typically support a wide range of input voltages, allowing flexibility in power supply options.

1.2.4. Reset Button: Arduino boards include a reset button, which can be pressed to restart the program running on the board or to enter programming mode for uploading new code.

1.2.5. Clock Crystal: Many Arduino boards include a crystal oscillator to provide precise timing for the microcontroller. This helps ensure accurate timing for time-sensitive operations and communication protocols.

1.2.6. Voltage Regulator: Arduino boards often feature a voltage regulator that regulates the incoming voltage to provide a stable supply voltage for the microcontroller and other components on the board.

1.2.7. Serial Interface: Arduino boards typically include a Universal Serial Bus (USB) interface for communication with a computer, allowing users to upload code to the board and monitor serial output from the board's microcontroller.

1.2.8. LEDs: Some Arduino boards include built-in LEDs that can be used for visual feedback or debugging purposes. For example, the Arduino Uno has a built-in LED connected to pin 13, which can be controlled programmatically.

1.2.9. Expansion Headers: Arduino boards often include headers or connectors for attaching expansion boards, also known as shields. Shields are add-on boards that extend the capabilities of the Arduino by adding functionality such as wireless communication, motor control, or additional sensors.

1.2.10. Integrated Development Environment (IDE): The Arduino IDE is a software application used for writing, compiling, and uploading code to Arduino boards. It provides a user-friendly interface for writing programs in the Arduino programming language, which is based on Wiring, a simplified version of C and C++.

Arduino boards are widely used in various applications, including robotics, home automation, Internet of Things (IoT), interactive art installations, and educational projects. Their simplicity, versatility, and affordability make them an excellent choice for hobbyists, students, and professionals alike who want to bring their electronic projects to life.

1.3 Arduino IDE & Arduino Droid

1.3.1. Arduino IDE (Integrated Development Environment):

- The Arduino IDE is a software application used for programming Arduino boards.

- It provides a user-friendly interface for writing, compiling, and uploading code to Arduino boards.

- The IDE supports the Arduino programming language, which is based on C and C++.

- It includes features like code highlighting, auto-completion, and serial monitor for debugging.

- The Arduino IDE is available for Windows, macOS, and Linux, making it accessible to a wide range of users.

1.3.2. Arduinodroid:

- Arduinodroid is an Android application that serves as an alternative to the Arduino IDE.

- It allows users to write, compile, and upload Arduino code directly from their Android devices.

- Arduinodroid provides a simplified interface optimized for touchscreen devices.

- While it doesn't offer all the features of the Arduino IDE, it provides essential functionalities for programming Arduino boards on the go.

- Arduinodroid is particularly useful for users who want to work with Arduino projects on their smartphones or tablets, without needing access to a computer.

In summary, the Arduino IDE is a comprehensive desktop application for programming Arduino boards, while Arduinodroid is a mobile app designed for programming Arduino boards directly from Android devices.

1.4 Latex Programming

LaTeX is a typesetting system commonly used for creating documents with complex formatting, such as academic papers, reports, theses, and presentations. It is widely used in academia, particularly in fields such as mathematics, computer science, engineering, and physics. LaTeX provides a high level of control over document layout and formatting, allowing users to focus on content creation while the system takes care of formatting.

Here's a detailed explanation of LaTeX programming:

1.4.1. Document Structure:

In LaTeX, documents are written in plain text format with a `.tex` extension. The basic structure of a LaTeX document includes:

- Document Class: Specifies the type of document being created (e.g., article, report, book, presentation).

- Preamble: Contains commands that set document-wide parameters, such as the font size, margins, and document title.

- Document Body: Contains the main content of the document, including text, sections, subsections, lists, tables, equations, and figures.

1.4.2. Commands and Environments:

LaTeX documents consist of commands and environments that control various aspects of document formatting and content. Commands typically start with a backslash `\`, followed by the command name and optional arguments enclosed in curly braces `{}`. For example, `\section{Introduction}` is a command that creates a section titled "Introduction". Environments define regions of the document where specific formatting rules apply. For example, the `equation` environment is used for displaying mathematical equations.

1.4.3. Packages:

LaTeX provides a basic set of functionality out of the box, but additional features can be added using packages. Packages extend LaTeX's capabilities by providing commands, environments, and formatting options for specific purposes. For example, the `graphicx` package allows for the inclusion of graphics in a document, while the `amsmath` package provides enhanced support for mathematical typesetting.

1.4.4. Mathematical Typesetting:

LaTeX excels at typesetting mathematical expressions and equations. It provides a comprehensive set of commands and environments for creating mathematical content, including fractions, exponents, integrals, matrices, and more. LaTeX's mathematical typesetting capabilities are widely used in academic writing, particularly in fields where mathematical notation is prevalent.

1.4.5. Cross-Referencing and Citations:

LaTeX makes it easy to cross-reference sections, figures, tables, and equations within a document using labels and references. Labels are assigned to specific elements in the document, and references are used to refer to those elements elsewhere in the document. LaTeX also supports bibliographies and citations through packages like `biblatex` and `natbib`, allowing users to manage citations and generate formatted bibliographies automatically.

1.4.6. Customization:

One of LaTeX's strengths is its high level of customization. Users can create custom commands, environments, and document classes to meet specific formatting requirements. LaTeX documents can be styled using predefined document classes or custom style files (`.sty`). Additionally, users can define their own macros and packages to streamline document creation and ensure consistency across multiple documents.

1.4.7. Compilation:

LaTeX documents must be compiled to produce output in the desired format (e.g., PDF). The compilation process typically involves running a LaTeX compiler (e.g., `pdflatex`, `xelatex`, `lualatex`) on the `.tex` source file. During compilation, LaTeX processes the source file, interprets commands and environments, resolves references, and generates the final output document.

In summary, LaTeX programming involves writing and formatting documents using LaTeX markup language, which provides extensive control over document layout, content, and formatting. With its powerful typesetting capabilities and flexibility, LaTeX is a popular choice for creating high-quality documents, particularly in academic and technical fields.

1.5 Neovim

Neovim is a highly configurable, modern text editor based on the popular Vim editor. It is designed to improve upon Vim by addressing some of its limitations and providing additional features, while maintaining compatibility with Vim's powerful editing capabilities and modal editing paradigm.

Here's a detailed explanation of Neovim's features and functionality:

1.5.1. Modal Editing:

Neovim, like Vim, follows a modal editing paradigm, where the keyboard behaves differently depending on the current mode. The primary modes in Neovim are:

- Normal Mode: Used for navigation, text manipulation, and executing commands.
- Insert Mode: Used for inserting text into the document.
- Visual Mode: Used for selecting blocks of text for manipulation.

- Command-Line Mode: Used for entering Ex commands, searching, and other advanced operations.

1.5.2. Extensibility:

Neovim is designed to be highly extensible, allowing users to customize and extend its functionality using plugins written in various programming languages, including Vimscript, Lua, Python, and others. Plugins can add new commands, mappings, syntax highlighting, autocompletion, and more, allowing users to tailor Neovim to their specific needs.

1.5.3. Built-In Features:

Neovim includes many features out of the box, including:

- Syntax Highlighting: Neovim provides syntax highlighting for a wide range of programming languages and file formats, making code easier to read and understand.

- Code Completion: Neovim offers built-in support for code completion, allowing users to quickly insert code snippets, function names, and variable names.

- Split Windows: Neovim supports splitting the editing window into multiple panes, allowing users to work on multiple files or sections of code simultaneously.

- File Navigation: Neovim includes commands and shortcuts for quickly navigating between files, directories, and buffers.

- Version Control Integration: Neovim integrates with version control systems like Git, providing commands and shortcuts for viewing diffs, committing changes, and navigating through version history.

1.5.4. Performance:

Neovim aims to improve upon Vim's performance and responsiveness, particularly for large files and long editing sessions. It achieves this through various optimizations, including asynchronous execution of plugins and built-in features, better memory management, and reduced startup times.

1.5.5. Remote Plugins and API:

Neovim introduces a remote plugin architecture, allowing external programs to communicate with Neovim over standard input/output channels. This enables the development of language servers, debugging tools, and other external integrations that can enhance Neovim's functionality

1.5.6. Community and Ecosystem:

Neovim has a vibrant community of users and developers who contribute plugins, themes, and enhancements to the editor. The Neovim community is active on forums, chat channels, and code repositories, providing support, sharing tips and tricks, and collaborating on improving the editor.

In summary, Neovim is a modern, extensible text editor that builds upon the foundation of Vim while introducing new features, optimizations, and extensibility mechanisms. With its focus on performance, extensibility, and compatibility with Vim, Neovim is a popular choice for developers and power users who require a highly customizable and efficient editing environment.

1.6 Ubuntu

Ubuntu is a widely used open-source operating system (OS) based on the Linux kernel. It is known for its ease of use, stability, and strong community support. Ubuntu is developed and maintained by Canonical Ltd., a company founded by South African entrepreneur Mark Shuttleworth. Here's a detailed explanation of Ubuntu:

1.6.1. History and Philosophy:

Ubuntu was first released in October 2004, with the goal of providing a free and user-friendly alternative to proprietary operating systems like Windows and macOS. The name "Ubuntu" is derived from an African philosophy meaning "humanity towards others." This reflects the project's commitment to open-source principles, collaboration, and community involvement.

1.6.2. Versions and Releases:

Ubuntu follows a time-based release schedule, with new versions released every six months. Each release is codenamed using an adjective and an animal name, such as "Bionic Beaver" or "Focal Fossa," following an alphabetical order. Every two years, a Long Term Support (LTS) version is released, which receives updates and support for five years, making it ideal for production environments and enterprise use.

1.6.3. Desktop Environment:

Ubuntu uses the GNOME desktop environment as its default user interface. GNOME provides a modern and intuitive desktop experience, with features like a customizable dock,

application launcher, and system tray. However, Ubuntu also supports other desktop environments such as KDE Plasma, Xfce, LXQt, and MATE, which users can install and choose from based on their preferences.

1.6.4. Package Management:

Ubuntu uses the Debian package management system, with the Advanced Package Tool (APT) as the primary package manager. Users can install, remove, and update software packages using command-line tools like `apt` or graphical package managers such as Ubuntu Software Center or Synaptic Package Manager. Ubuntu's vast software repositories offer thousands of free and open-source applications that can be easily installed with a few clicks.

1.6.5. Security:

Security is a top priority for Ubuntu, with regular security updates and patches provided to address vulnerabilities and ensure system integrity. Ubuntu includes features like AppArmor, a mandatory access control framework, to confine and protect applications from potential threats. Additionally, Ubuntu benefits from the robust security features inherent in Linux, such as user permissions, encryption, and firewall capabilities.

1.6.6. Server Edition:

In addition to the desktop version, Ubuntu offers a server edition tailored for server deployments and cloud environments. Ubuntu Server includes features like the Ubuntu Advantage service for enterprise support, integration with cloud platforms like Amazon Web Services (AWS) and Microsoft Azure, and specialized tools for system administration, virtualization, containerization, and automation.

1.6.7. Community and Support:

Ubuntu has a large and active community of users, developers, and contributors who collaborate on development, provide support, and contribute to the ecosystem. The Ubuntu community offers forums, mailing lists, IRC channels, and online resources where users can seek help, share knowledge, and participate in discussions. Canonical also provides commercial support options for organizations and businesses through the Ubuntu Advantage program.

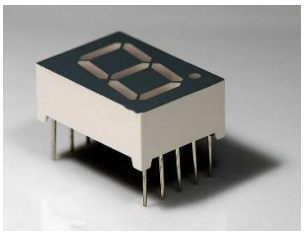
Overall, Ubuntu is a versatile and user-friendly operating system that caters to a wide range of users, from desktop enthusiasts to enterprise IT professionals. With its commitment to open-source principles, regular updates, extensive software ecosystem, and strong community support, Ubuntu continues to be a popular choice for individuals, organizations, and institutions around the world.

CHAPTER 2

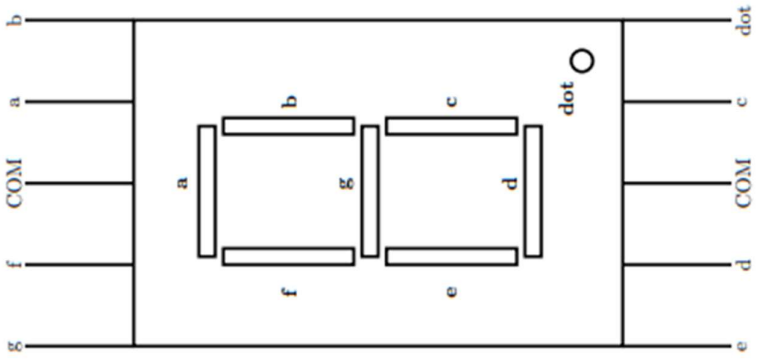
DIGITAL DESIGN TASKS

2.1 Seven Segment

A seven-segment display is a form of electronic display device used to represent decimal numbers. It consists of seven individual LED (light-emitting diode) segments arranged in a specific pattern to form numerals from 0 to 9. Each segment is capable of being independently activated, allowing the display of various combinations to represent different numbers. The seven segments are arranged in a pattern resembling the number "8," with an additional segment on the top, bottom, middle, and each side.



The seven segments are typically labeled as 'a', 'b', 'c', 'd', 'e', 'f', and 'g'. Here's a basic representation of how these segments are arranged:



Each segment represents a part of the digit to be displayed. When certain segments are activated or deactivated, they form different numbers. For example:

- To display the number 0, segments 'a', 'b', 'c', 'd', 'e', and 'f' are illuminated, while 'g' remains off.

- To display the number 1, segments 'b' and 'c' are illuminated, while the rest are off.

- And so on for numbers 2 through 9.

In addition to displaying decimal numbers, seven-segment displays can also show some alphabetic characters, such as 'A', 'b', 'C', 'd', 'E', 'F', 'g', 'H', 'L', 'P', and 't'. However, these

representations are not as common and typically require additional segments to display accurately.

Seven-segment displays are commonly used in various electronic devices where numerical information needs to be displayed, such as digital clocks, calculators, electronic meters, and many more. They are popular due to their simplicity, low cost, and ease of use in displaying numerical data.

There are different types of seven-segment displays, including common anode and common cathode configurations. In a common anode display, all the anode connections of the LED segments are tied together and connected to a positive voltage source, while each cathode connection represents an individual segment. In a common cathode display, it's the opposite: all cathodes are tied together and connected to ground, while each anode connection represents an individual segment.

Driving a seven-segment display typically requires a microcontroller or dedicated driver IC to control the illumination of each segment. The microcontroller sends signals to the display to activate the desired segments corresponding to the number to be displayed. Multiplexing techniques are often used to drive multiple seven-segment displays using fewer pins on the microcontroller.

2.2 7447

The 7447 IC is a commonly used integrated circuit (IC) that serves as a BCD (Binary-Coded Decimal) to 7-segment decoder/driver. It takes a BCD input (typically a 4-bit binary input) and drives a 7-segment display to show the corresponding decimal digit.

Here's a breakdown of its functionality:

2.2.1. BCD Input: The 7447 IC typically accepts a 4-bit BCD input, representing a decimal digit from 0 to 9. BCD is a binary representation of decimal numbers, where each digit is represented by its corresponding 4-bit binary code.

2.2.2. Decoder: The IC internally decodes the BCD input and activates the appropriate segments of a connected 7-segment display to represent the corresponding decimal digit. For example, if the input is '0000', representing the decimal digit 0, it activates the segments required to display '0' on the 7-segment display.

2.2.3. Driver: The IC also includes current-limiting resistors on its output pins to drive the segments of the 7-segment display directly. This means that the IC can interface directly with a 7-segment display without requiring additional components to control the display.

2.2.4. 7-Segment Output: The output pins of the 7447 IC correspond to the segments ('a' through 'g') of a typical 7-segment display. When a particular decimal digit is decoded, the corresponding segments are activated to display that digit on the 7-segment display.

2.2.5. Decimal Point (Dot) Output: Some versions of the 7447 IC also include an output for the decimal point ('DP') segment of the 7-segment display, allowing for the display of decimal numbers with fractional parts

2.2.6. Common Anode/Cathode Configuration: The 7447 IC is available in variants for both common anode and common cathode 7-segment displays. The choice depends on the type of 7-segment display being used in the circuit.

2.2.7. Binary to Decimal Conversion: The 7447 IC simplifies the process of converting binary-coded decimal numbers to decimal digits for display on 7-segment displays, making it a convenient component in various digital display applications such as digital clocks, calculators, and electronic meters.

It's important to note that while the 7447 IC is widely used and highly convenient for driving 7-segment displays, it is an older IC and may be less common in modern designs. However, its functionality can often be replicated using microcontrollers or more modern display driver ICs.

2.3 K – Map

Karnaugh Maps, often abbreviated as K-Maps, are a graphical method used to simplify Boolean algebra expressions. They are particularly useful for reducing logic circuits and minimizing the number of logic gates required to implement a digital logic function. K-Maps provide a systematic approach to finding the simplest form of a Boolean expression by visually organizing and grouping terms with similar characteristics.

Here's a detailed breakdown of how Karnaugh Maps work:

2.3.1. Grid Representation:

- Karnaugh Maps are represented as grids, with each cell in the grid corresponding to a unique combination of input variables.

- The number of rows and columns in the grid depends on the number of input variables. For example, with two input variables (A and B), the K-Map is a 2x2 grid. With three input variables (A, B, and C), it's a 2x4 grid, and so on.

2.3.2. Gray Code Ordering:

- The cells in a Karnaugh Map are filled in a specific order called Gray code. Gray code ensures that adjacent cells differ by only one variable. This helps in identifying adjacent cells that can be grouped together during simplification.

2.3.3. Grouping:

- The main simplification technique in Karnaugh Maps involves grouping together adjacent cells that contain '1's.

- Groups can be formed horizontally, vertically, or in a combination of both. Each group should contain 1, 2, 4, 8, etc., adjacent cells (1, 2, 4, 8, 16, etc., depending on the number of input variables).

2.3.4. Minimal Sum-of-Products Expression:

- After grouping adjacent cells, the goal is to cover as many '1's as possible with the fewest number of groups.

- Each group corresponds to a term in the simplified Boolean expression.

- Once groups are identified, each group can be translated into a product term.

- Finally, these product terms are combined using logical OR operations to form the simplified Boolean expression.

Example:

Let's consider a simple example with a 2-variable Karnaugh Map:

```
    B\A  00  01  11  10
      +---+---+---+---+
     00| 0 | 1 | 0 | 1 |
      +---+---+---+---+
     01| 1 | 0 | 1 | 0 |
      +---+---+---+---+
```

In this example, the '1's are grouped into two groups: one group of two adjacent cells and one group of four adjacent cells. This leads to a simplified expression of:

```
F(A, B) = A' + B'
```

Advantages of Karnaugh Maps:

1. Visual Representation: K-Maps provide a visual aid, making it easier to understand and manipulate Boolean expressions.

2. Systematic Approach: The systematic approach of grouping adjacent cells simplifies the process of Boolean expression minimization.

3. Guaranteed Optimality: K-Maps guarantee the optimal solution for minimizing Boolean expressions.

Limitations:

1. Limited to a Small Number of Variables: As the number of variables increases, Karnaugh Maps become more complex and cumbersome.

2. Manual Process: While K-Maps are effective for small to moderately sized problems, they can become impractical for larger circuits.

3. Subjective: Grouping cells can sometimes be subjective, leading to different possible simplifications

Despite these limitations, Karnaugh Maps remain a valuable tool in the toolbox of digital logic designers for simplifying Boolean expressions and optimizing logic circuits.

2.4 7474

The 7474 is a widely used integrated circuit (IC) chip that contains two identical D-type flip-flops. Each flip-flop is capable of storing a single bit of data. The 7474 is a member of the 7400 series of TTL (Transistor-Transistor Logic) ICs, which were commonly used in digital electronics before the widespread adoption of CMOS (Complementary Metal-Oxide-Semiconductor) technology.

Basic Functionality:

2.4.1. Flip-Flop Operation:

- Each flip-flop within the 7474 is a D-type flip-flop, which means it has a data (D) input, a clock (CLK) input, a set (S) input, a reset (R) input, and complementary outputs (Q and Q\).

- When the clock input (CLK) transitions from low to high (rising edge), the D input is sampled and transferred to the output Q. The output Q holds the state of the D input until the next clock edge.

- The complementary output (Q\) is the inverse of Q.

2.4.2. Set (S) and Reset (R) Inputs:

- The set (S) input forces the Q output to logic high (1), regardless of the state of other inputs, when activated.

- The reset (R) input forces the Q output to logic low (0), regardless of the state of other inputs, when activated.

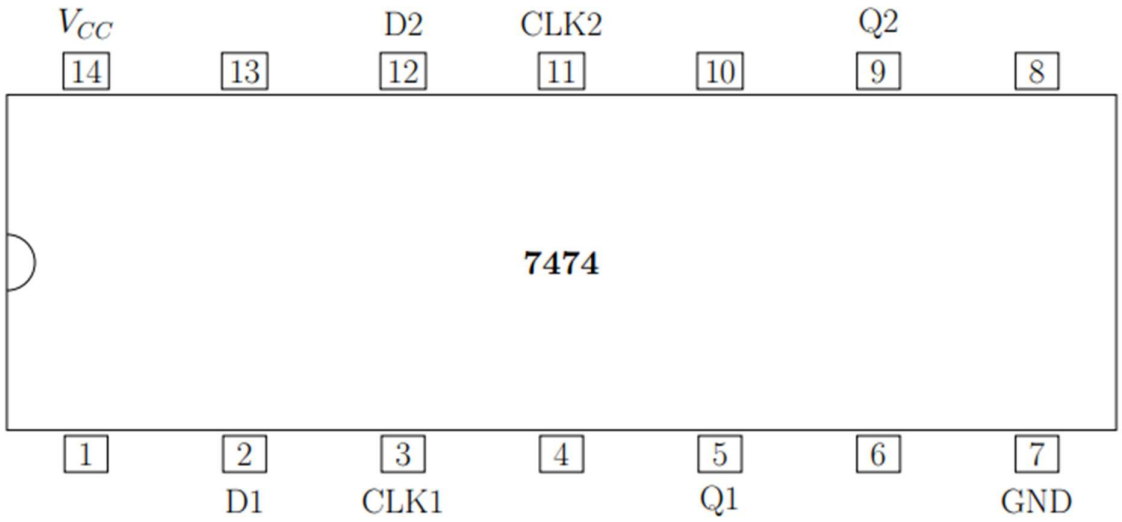
2.4.3. Dual Flip-Flop Configuration:

- The 7474 IC contains two independent D-type flip-flops, allowing it to store and manipulate two separate bits of data simultaneously.

- Each flip-flop operates independently of the other, with its own set of inputs and outputs.

Pin Configuration (Typical):

- Pin 1: Data (D) Input for Flip-Flop 1
- Pin 2: Clock (CLK) Input for Flip-Flop 1
- Pin 3: Set (S) Input for Flip-Flop 1
- Pin 4: Complementary Output (Q\) for Flip-Flop 1
- Pin 5: Output (Q) for Flip-Flop 1
- Pin 6: Reset (R) Input for Flip-Flop 1
- Pin 7: Reset (R) Input for Flip-Flop 2
- Pin 8: Data (D) Input for Flip-Flop 2
- Pin 9: Clock (CLK) Input for Flip-Flop 2
- Pin 10: Set (S) Input for Flip-Flop 2
- Pin 11: Complementary Output (Q\) for Flip-Flop 2
- Pin 12: Output (Q) for Flip-Flop 2
- Pin 13: GND (Ground)
- Pin 14: Output (Q) for Flip-Flop 2
- Pin 15: Output (Q) for Flip-Flop 1
- Pin 16: Vcc (Power Supply)



Applications:

- Digital Counters: The 7474 can be used as a building block for constructing various types of digital counters, including binary counters and ripple counters.

- Sequential Logic Circuits: It is commonly used in sequential logic circuits for storing and transferring data between different stages.

- Frequency Dividers: By using multiple 7474 ICs cascaded together, frequency dividers can be implemented to divide the frequency of an input signal.

- Clocking and Synchronization: It is used in clocked systems to synchronize operations and ensure proper timing between different components.

Advantages:

- Ease of Use: The 7474 is a straightforward IC with clear functionality, making it easy to integrate into digital circuits.

- Dual Flip-Flops: Its dual flip-flop configuration provides versatility and saves board space in designs requiring multiple flip-flops.

- Compatibility: It operates on standard TTL logic levels, making it compatible with other TTL ICs and digital systems.

Limitations:

- Power Consumption: As a TTL IC, the 7474 can consume relatively high power compared to modern CMOS equivalents.

- Speed: It may not be suitable for high-speed applications due to its propagation delay and setup/hold time requirements.

- Obsolete: While still available, the 7474 is considered an older technology, and newer designs may opt for more modern flip-flop ICs or programmable logic devices.

In summary, the 7474 is a versatile and widely used IC in digital electronics, particularly in applications requiring flip-flops for sequential logic operations. Its simplicity, dual flip-flop configuration, and compatibility with TTL logic make it a valuable component in various digital designs.

2.5 State Machine

A state machine, also known as a finite state machine (FSM), is a mathematical model used in computer science and engineering to represent the behavior of a system that can be in a finite number of states at any given time. State machines are widely used in various applications, including digital circuit design, software engineering, protocol specifications, and more.

Components of a State Machine:

1. States (S):

- States represent the different modes or conditions that a system can be in at any given time.

- Each state typically represents a specific configuration or behavior of the system.

- States are usually denoted by unique labels or identifiers.

2. Transitions (T):

- Transitions represent the events or conditions that cause the system to change from one state to another.

- Transitions are triggered by external inputs, internal conditions, or the passage of time.

- Transitions are typically represented by directed edges between states.

3. Inputs (I):

- Inputs are external signals or events that trigger transitions between states.

- Inputs can be physical signals, such as sensor readings, user inputs, or communication messages.

- Each transition may be associated with specific input conditions that must be met for the transition to occur.

4. Outputs (O):

- Outputs are signals or actions produced by the system in response to inputs or state changes.

- Outputs can include actions taken by the system, data sent to other components, or signals indicating the system's current state.

- Outputs may be associated with specific states or transitions.

Types of State Machines:

1. Finite State Machine (FSM):

- A finite state machine has a finite number of states, transitions, inputs, and outputs.

- FSMs are well-suited for modeling systems with discrete, deterministic behavior.

2. Mealy Machine:

- In a Mealy machine, outputs are dependent on both the current state and the inputs that trigger transitions.

- The output function is associated with transitions between states.

3. Moore Machine:

- In a Moore machine, outputs are dependent only on the current state and not on the inputs that trigger transitions.

- The output function is associated with individual states.

4. Hierarchical State Machine:

- A hierarchical state machine consists of multiple levels of state machines, with higher-level states controlling the behavior of lower-level states.

- Hierarchical state machines are useful for modeling complex systems with nested behaviors.

Applications of State Machines:

1. Digital Circuit Design:

- State machines are used to design digital circuits, such as counters, sequencers, and controllers.

- They can be implemented using hardware components like flip-flops and combinational logic gates.

2. Software Engineering:

- State machines are used to model and implement the behavior of software systems, including embedded systems, user interfaces, and protocol handlers.

- They can be implemented using programming constructs like switch-case statements or object-oriented design patterns.

3. Protocol Specifications:

- State machines are used to specify the behavior of communication protocols, such as network protocols, data transfer protocols, and control protocols.

- They provide a formal and concise representation of the protocol's behavior.

4. Game Development:

- State machines are used to model the behavior of characters, game states, and game mechanics in video game development.

- They help manage the complexity of game logic and interaction.

Advantages of State Machines:

1. Modularity:

- State machines help break down complex systems into simpler, manageable components.

- Each state and transition can be designed and tested independently.

2. Clarity and Understandability:

- State machines provide a visual and intuitive representation of system behavior.

- They make it easier to understand and reason about the behavior of a system.

3. Flexibility:

- State machines can be easily modified and extended to accommodate changes in system requirements.

- New states and transitions can be added, and existing ones can be modified or removed.

4. Verification and Testing:

- State machines facilitate formal verification and testing of system behavior.

- Test cases can be systematically derived from state transitions to ensure full coverage of system functionality.

In summary, state machines are powerful tools for modeling and designing systems with discrete, sequential behavior. They provide a structured approach to capturing and analyzing system behavior, making them invaluable in a wide range of applications across various domains.

2.6 PlatformIO

PlatformIO is an open-source ecosystem for embedded development that provides a unified platform for writing, building, and debugging firmware for embedded systems. It offers a variety of features and tools to streamline the development process, including support for multiple hardware platforms, libraries, and development environments. Here's an in-depth explanation of PlatformIO:

Key Components and Features:

1. Unified Development Environment:

- PlatformIO provides a unified development environment that supports multiple integrated development environments (IDEs), including Visual Studio Code (VS Code), Atom, and JetBrains IDEs like CLion and IntelliJ IDEA.

- Developers can choose their preferred IDE and use PlatformIO's extension or plugin to access its features seamlessly.

2. Cross-Platform Support:

- PlatformIO supports a wide range of hardware platforms, including popular microcontrollers such as Arduino, ESP8266, ESP32, STM32, AVR, and more.

- It also supports various development boards and modules from different manufacturers, enabling developers to work with diverse hardware environments.

3. Package Management:

- PlatformIO provides a package management system that allows developers to easily install, manage, and share libraries, frameworks, and toolchains.

- Libraries and frameworks can be installed directly from the PlatformIO Library Manager or via PlatformIO Core, the command-line interface.

4. Built-in Build System:

- PlatformIO includes a powerful build system that automatically resolves dependencies, compiles source code, and generates firmware binaries for the target platform.

- It supports both local and remote builds, enabling developers to compile code locally or use cloud-based build services for faster compilation.

5. Integrated Debugger:

- PlatformIO offers integrated debugging features for hardware debugging using popular debugging tools such as GDB (GNU Debugger) and OpenOCD (Open On-Chip Debugger).

- Developers can debug their firmware directly from the IDE, set breakpoints, inspect variables, and step through code execution.

6. Unit Testing:

- PlatformIO supports unit testing of firmware using popular testing frameworks such as Unity, Google Test, and Catch.

- Developers can write and execute unit tests to verify the functionality and reliability of their firmware code.

7. Continuous Integration (CI) Integration:

- PlatformIO integrates seamlessly with continuous integration (CI) platforms like Travis CI, CircleCI, and Jenkins.

- Developers can automate the build and testing process, ensuring that changes to the firmware are tested and validated automatically

Workflow with PlatformIO:

1. Project Initialization:

- Developers can initialize a new project using the PlatformIO command-line interface (CLI) or through the IDE's graphical interface.

- PlatformIO generates project files and directories, including configuration files (platformio.ini), source code files, and build artifacts.

2. Coding and Development:

- Developers write firmware code using their preferred text editor or IDE, leveraging PlatformIO's features such as code completion, syntax highlighting, and linting.

- They can include libraries and frameworks using the PlatformIO Library Manager or by specifying dependencies in the project configuration file.

3. Building and Compilation:

- Developers compile the firmware code using PlatformIO's build system, which resolves dependencies, compiles source files, and generates firmware binaries.

- Compilation output, including errors and warnings, is displayed in the IDE's console or terminal window.

4. Debugging and Testing:

- Developers debug and test their firmware using PlatformIO's integrated debugging tools and unit testing frameworks.

- They can set breakpoints, inspect variables, and execute unit tests to verify the functionality and correctness of their code.

5. Deployment and Deployment:

- Once the firmware is tested and validated, developers can deploy it to the target hardware platform using PlatformIO's upload functionality.

- PlatformIO handles the flashing process, transferring the firmware binary to the target device via USB, serial, or network connection.

Advantages of PlatformIO:

1. Ease of Use: PlatformIO simplifies the development process by providing a unified platform with integrated tools and features.

2. Flexibility: PlatformIO supports a wide range of hardware platforms, IDEs, and programming languages, giving developers flexibility in their development workflow.

3. Community and Ecosystem: PlatformIO has a vibrant community of developers and contributors who share libraries, frameworks, and knowledge.

4. Productivity: PlatformIO's automation features, such as dependency resolution and continuous integration, help improve productivity and efficiency in embedded development projects

In summary, PlatformIO is a versatile and powerful platform for embedded development, offering a unified environment with integrated tools and features to streamline the development process. Its cross-platform support, package management system, and built-in build system make it a valuable tool for developers working on embedded systems projects.

2.7 AVRA Assembly

AVRA is a free open-source cross-platform assembler for Atmel's AVR microcontrollers. It allows developers to write assembly language programs for AVR microcontrollers and generate machine code binaries that can be flashed onto AVR devices using tools like AVRDUDE. AVRA is commonly used in embedded systems development, particularly for projects that require low-level control and optimization.

Here's a detailed explanation of AVRA Assembly:

Assembly Language:

Assembly language is a low-level programming language that provides a symbolic representation of machine code instructions. Each assembly language instruction corresponds directly to a machine instruction understood by the processor. Assembly language programs are typically written using mnemonics that represent the operations performed by the CPU, along with operands that specify data or memory addresses.

AVRA Features:

1. Support for AVR Microcontrollers:

- AVRA is specifically designed to work with Atmel's AVR microcontrollers, including popular chips like ATmega, ATtiny, and ATxmega series.

- It supports the full range of AVR instruction set architectures (ISAs) and peripherals, allowing developers to take full advantage of AVR microcontroller features.

2. Cross-Platform Compatibility:

- AVRA is available for multiple operating systems, including Windows, Linux, and macOS, making it accessible to a wide range of developers.

- It can be easily installed and used on different development environments and toolchains.

3. Optimization and Code Generation:

- AVRA includes optimization features to generate efficient machine code from assembly language source files.

- It supports various output formats, including Intel Hex and binary formats, suitable for flashing onto AVR microcontrollers using programming tools.

4. Integration with AVRDUDE:

- AVRDUDE is a utility for programming AVR microcontrollers that supports a wide range of programming hardware and protocols.

- AVRA integrates seamlessly with AVRDUDE, allowing developers to flash the generated machine code binaries onto AVR devices directly from the command line.

5. Macro Support:

- AVRA provides support for macros, which allow developers to define reusable code snippets and simplify complex assembly language constructs.

- Macros enhance code readability, maintainability, and reusability, reducing the overall development time and effort.

6. Error Checking and Debugging:

- AVRA performs syntax checking and error detection during the assembly process, providing helpful error messages and warnings to aid debugging.

- Developers can easily identify and fix syntax errors, typos, and other issues in their assembly language code.

Workflow with AVRA:

1. Writing Assembly Language Code:

- Developers write assembly language programs using a text editor or integrated development environment (IDE), using AVRA's syntax and mnemonics.

- Assembly language code typically consists of instructions, labels, constants, and comments.

2. Compiling with AVRA:

- Developers compile the assembly language source files using AVRA's command-line interface or integration with IDEs.

- AVRA translates the assembly language source code into machine code binaries, optimized for the target AVR microcontroller.

3. Flashing onto AVR Microcontrollers:

- Once the machine code binaries are generated, developers use AVRDUDE or other programming tools to flash the binaries onto AVR microcontrollers.

- AVRDUDE communicates with the AVR microcontroller's programming interface (e.g., ISP, JTAG, UART) to upload the firmware.

Advantages of AVRA Assembly

1. Low-Level Control: AVRA allows developers to directly manipulate AVR microcontroller hardware and peripherals, providing precise control over system behavior.

2. Performance Optimization: AVRA's optimization features help generate efficient machine code, maximizing the performance of AVR microcontroller applications.

3. Platform Compatibility: AVRA's cross-platform compatibility ensures that developers can work on different operating systems without any limitations.

4. Community Support: AVRA benefits from a supportive community of AVR enthusiasts and developers who contribute libraries, tutorials, and resources.

In summary, AVRA Assembly provides a powerful and flexible tool for writing assembly language programs for AVR microcontrollers. Its optimization features, cross-platform compatibility, and integration with AVRDUDE make it a valuable asset for embedded systems development with AVR microcontrollers.

2.8 AVR – GCC

AVR-GCC is a compiler toolchain for Atmel AVR microcontrollers, which is part of the GNU Compiler Collection (GCC). It allows developers to write, compile, and debug C and C++ programs for AVR microcontrollers. Here's a brief explanation:

Components of AVR-GCC:

1. Compiler (avr-gcc):

- The avr-gcc compiler translates C and C++ source code into machine code (assembly language) specific to AVR microcontrollers.

- It supports the full range of C and C++ language features, including standard libraries and optimizations.

2. Assembler (avr-as):

- The avr-as assembler converts assembly language source code into machine code object files (.o) that can be linked with other object files.

- It translates mnemonic instructions into binary machine code understood by AVR microcontrollers.

3. Linker (avr-ld):

- The avr-ld linker combines multiple object files (.o) generated by the compiler and assembler into a single executable program.

- It resolves symbols, manages memory layout, and generates the final output file in ELF or Intel Hex format.

4. Standard Libraries:

- AVR-GCC provides a set of standard libraries (avr-libc) specifically tailored for AVR microcontrollers.

- These libraries include functions and macros for accessing hardware peripherals, performing common tasks, and interfacing with external devices.

5. Header Files:

- AVR-GCC includes header files that define register and bit-level access macros for AVR microcontroller peripherals.

- These header files provide developers with a standardized interface for interacting with hardware, simplifying the development process.

Workflow with AVR-GCC:

1. Writing Source Code:

- Developers write C or C++ source code for AVR microcontrollers using a text editor or integrated development environment (IDE).

- They leverage AVR-GCC's standard libraries and header files to access hardware peripherals and features.

2. Compiling with AVR-GCC:

- Developers compile the source code using the avr-gcc compiler, specifying the target AVR microcontroller and compilation options.

- AVR-GCC translates the source code into machine code object files (.o) optimized for the AVR architecture.

3. Linking Object Files:

- Developers link the object files generated by the compiler with any necessary libraries using the avr-ld linker.

- AVR-GCC resolves symbols, performs memory allocation, and generates the final executable program in ELF or Intel Hex format.

4. Flashing onto AVR Microcontrollers:

- Once the executable program is generated, developers use programming tools like AVRDUDE to flash the program onto AVR microcontrollers.

- AVRDUDE communicates with the AVR microcontroller's programming interface (e.g., ISP, JTAG, UART) to upload the firmware.

Advantages of AVR-GCC:

1. Open-Source and Free: AVR-GCC is open-source software distributed under the GNU General Public License (GPL), making it freely available for anyone to use and modify.

2. Compatibility: AVR-GCC is cross-platform and compatible with various operating systems, including Linux, macOS, and Windows.

3. Community Support: AVR-GCC benefits from a large and active community of developers who contribute libraries, tutorials, and resources.

4. Optimization: AVR-GCC includes optimization features that help generate efficient machine code for AVR microcontrollers, maximizing performance and minimizing code size.

In summary, AVR-GCC is a powerful and versatile compiler toolchain for Atmel AVR microcontrollers, providing developers with the tools they need to write, compile, and debug C and C++ programs for embedded systems. Its open-source nature, compatibility, and optimization features make it a popular choice for AVR microcontroller development.

2.9 ESP32

The ESP32 is a highly versatile and powerful microcontroller developed by Espressif Systems. It is widely used in various IoT (Internet of Things) applications, including home automation, industrial automation, wearable devices, and more. Here's a brief overview of the ESP32:

Key Features:

1. Dual-Core Processor:

- The ESP32 features a dual-core Tensilica Xtensa LX6 processor, which allows for multitasking and improved performance in complex applications.

- Each core can run at speeds up to 240 MHz.

2. Wireless Connectivity:

- The ESP32 integrates Wi-Fi (802.11 b/g/n) and Bluetooth (Bluetooth Classic and Bluetooth Low Energy) connectivity, making it suitable for a wide range of wireless applications.

- It supports a variety of wireless protocols and standards, including TCP/IP, HTTP, MQTT, and more.

3. Rich Peripheral Set:

- The ESP32 features a rich set of peripherals, including GPIO pins, SPI, I2C, UART, ADC, DAC, PWM, and more.

- It also includes hardware cryptographic accelerators, which provide secure communication and encryption capabilities.

4. Low Power Consumption:

- The ESP32 is designed for low power consumption, making it suitable for battery-powered and energy-efficient applications.

- It supports various power-saving modes, including deep sleep mode, to minimize power consumption during idle periods.

5. Integrated Sensors and Peripherals:

- Some ESP32 modules come with integrated sensors and peripherals, such as temperature sensors, touch sensors, and Hall effect sensors.

- These built-in features enhance the functionality and versatility of the ESP32 for different applications.

6. Flexible Development Environment:

- The ESP32 can be programmed using various development frameworks and languages, including Arduino IDE, ESP-IDF (Espressif IoT Development Framework), MicroPython, and more.

- It offers flexibility for developers to choose the development environment that best suits their preferences and project requirements.

Applications:

1. IoT Devices:

- The ESP32 is widely used in IoT devices, such as smart home devices, environmental sensors, wearable devices, and connected appliances.

- Its wireless connectivity and low power consumption make it suitable for IoT applications that require remote monitoring and control.

2. Industrial Automation:

- The ESP32 can be used in industrial automation applications, including monitoring and control systems, data logging, and machine-to-machine communication.

- Its robustness, reliability, and support for industrial protocols make it suitable for deployment in industrial environments.

3. Consumer Electronics:

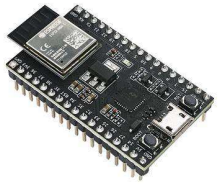
- The ESP32 is used in various consumer electronics products, including smartwatches, fitness trackers, remote controls, and audio devices.

- Its small form factor, low cost, and wireless connectivity make it an attractive choice for consumer electronics manufacturers.

4. Educational and Hobby Projects:

- The ESP32 is popular among hobbyists, makers, and educators for educational projects, prototyping, and DIY electronics.

- Its ease of use, affordability, and extensive documentation make it accessible to beginners and experienced developers alike.



Advantages

1. Versatility: The ESP32 is highly versatile, with support for a wide range of applications and development environments.

2. Cost-Effective: The ESP32 offers a cost-effective solution for adding wireless connectivity and advanced features to IoT devices and other embedded systems.

3. Community Support: The ESP32 has a large and active community of developers who contribute libraries, tutorials, and resources.

4. Integration: The ESP32 integrates multiple functionalities (processor, Wi-Fi, Bluetooth, etc.) into a single chip, simplifying system design and reducing component count.

Limitations:

1. Complexity: Developing complex applications for the ESP32 may require a good understanding of embedded systems and wireless protocols.

2. Documentation: While the ESP32 has extensive documentation, some advanced features and functionalities may require additional research and experimentation.

3. Power Consumption: While the ESP32 is designed for low power consumption, achieving optimal power efficiency in some applications may require careful optimization and tuning.

In summary, the ESP32 is a versatile and powerful microcontroller that offers a wide range of features and capabilities for IoT, industrial, consumer electronics, and educational applications. Its combination of wireless connectivity, rich peripherals, low power consumption, and flexibility make it a popular choice for embedded systems development.

2.10 ARM

ARM, originally Acorn RISC Machine, is a family of Reduced Instruction Set Computing (RISC) architectures for computer processors, developed by ARM Holdings (now part of NVIDIA). ARM processors are widely used in a variety of computing devices, ranging from

mobile phones and tablets to embedded systems, IoT devices, servers, and supercomputers. Here's a brief overview of ARM:

Key Features:

1. RISC Architecture:

- ARM processors are based on a Reduced Instruction Set Computing (RISC) architecture, which emphasizes simplicity and efficiency.

- RISC architectures typically have a small set of simple instructions, leading to faster execution and lower power consumption compared to Complex Instruction Set Computing (CISC) architectures.

2. Energy Efficiency:

- ARM processors are known for their energy efficiency, making them well-suited for battery-powered devices and embedded systems.

- ARM's emphasis on power efficiency has led to the widespread adoption of ARM-based processors in mobile devices and IoT applications.

3. Performance Scalability:

- ARM processors are highly scalable, with a wide range of performance levels catering to different application requirements.

- ARM offers various processor cores optimized for different use cases, from low-power microcontrollers to high-performance server-grade processors.

4. Instruction Set Architecture (ISA):

- ARM processors support multiple instruction set architectures, including ARMv7, ARMv8, and ARMv9.

- The ARMv8-A architecture, in particular, introduces features like 64-bit addressing, improved security, and support for virtualization, making it suitable for a broader range of applications, including servers and data centers.

5. Ecosystem and Licensing:

- ARM operates a licensing model, where it licenses its processor designs to semiconductor companies and system-on-chip (SoC) manufacturers.

- This licensing model has fostered a rich ecosystem of ARM-based products and solutions, with numerous companies developing custom ARM-based chips tailored to specific applications.

6. Wide Adoption:

- ARM processors are used in billions of devices worldwide, powering smartphones, tablets, smartwatches, IoT devices, automotive systems, networking equipment, and more.

- Their widespread adoption is driven by factors such as energy efficiency, performance, scalability, and the flexibility of the ARM architecture.

Applications:

1. Mobile Devices:

- ARM processors dominate the mobile device market, powering smartphones, tablets, and wearable devices.

- Their low power consumption, high performance, and support for multimedia applications make them ideal for mobile computing.

2. Embedded Systems:

- ARM processors are widely used in embedded systems, such as industrial automation, automotive electronics, home appliances, and consumer electronics.

- Their versatility, energy efficiency, and rich peripheral support make them suitable for a wide range of embedded applications.

3. Servers and Data Centers:

- ARM-based processors are gaining traction in the server and data center market, offering energy-efficient alternatives to traditional x86 processors.

- ARM servers are particularly well-suited for workloads that benefit from high-density deployments and energy efficiency, such as web hosting, cloud computing, and edge computing.

4. IoT and Edge Devices:

- ARM processors are commonly used in IoT devices and edge computing applications, providing the processing power and connectivity required for collecting, processing, and analyzing sensor data at the network edge.

Advantages:

1. Energy Efficiency: ARM processors offer excellent energy efficiency, making them suitable for battery-powered devices and energy-conscious applications.

2. Scalability: ARM processors are highly scalable, with a wide range of performance levels and configurations catering to diverse application requirements.

3. Ecosystem: ARM's licensing model has fostered a rich ecosystem of products, tools, and support services, enabling rapid innovation and customization.

4. Performance: ARM processors offer competitive performance levels across a range of applications, from low-power microcontrollers to high-performance servers.

Limitations:

1. Complexity: Developing software for ARM processors may require familiarity with ARM's architecture and tools, which can be challenging for some developers.

2. Compatibility: Porting software from x86-based systems to ARM-based systems may require modification or optimization to ensure compatibility and performance.

3. Vendor Lock-in: Choosing a specific ARM-based processor or vendor may lead to vendor lock-in, limiting flexibility and portability across different platforms.

In summary, ARM is a leading architecture for computer processors, offering energy-efficient, scalable, and versatile solutions for a wide range of applications. Its widespread adoption and strong ecosystem make it a popular choice for developers and manufacturers looking to build efficient and powerful computing devices.

2.11 FPGA

FPGA, or Field-Programmable Gate Array, is a type of integrated circuit (IC) that can be configured by a designer after manufacturing. Unlike traditional fixed-function ICs like microprocessors or memory chips, FPGAs can be reprogrammed or "reconfigured" to implement custom digital circuits, allowing for hardware customization and acceleration.

Key Components and Features:

1. Logic Blocks:

- FPGAs consist of an array of logic blocks, each containing configurable logic gates (such as AND, OR, and XOR gates), flip-flops, and other elements.

- These logic blocks can be interconnected to create complex digital circuits, including combinational logic, sequential logic, and arithmetic units.

2. Interconnects:

- The logic blocks in an FPGA are connected through a network of programmable interconnects, which allows signals to be routed between different blocks.

- The interconnects can be configured to implement various routing configurations, enabling flexible connectivity between logic blocks.

3. Configuration Memory:

- FPGAs contain configuration memory (usually in the form of static random-access memory or flash memory) that stores the logic configuration bitstream.

- The configuration bitstream specifies how the logic blocks and interconnects should be programmed to implement the desired digital circuit.

4. Clock Management:

- FPGAs typically include dedicated clock management resources, such as phase-locked loops (PLLs) and delay-locked loops (DLLs), for generating and distributing clock signals.

- These clock management resources ensure precise timing and synchronization within the FPGA design.

5. I/O Blocks:

- FPGAs feature input/output (I/O) blocks that provide interfaces for connecting external devices and peripherals.

- These I/O blocks support various standards and protocols, including LVCMOS, LVDS, DDR, PCIe, Ethernet, and more.

Advantages of FPGAs:

1. Flexibility and Customization:

- FPGAs offer unparalleled flexibility and customization, allowing designers to implement custom digital circuits tailored to specific application requirements.

- They are ideal for prototyping, rapid development, and iterative design cycles, as changes can be made quickly without the need for physical hardware modifications.

2. Parallelism and Acceleration:

- FPGAs excel at parallel processing and can perform multiple tasks simultaneously, making them well-suited for compute-intensive and parallelizable algorithms.

- They are commonly used to accelerate specific tasks in applications such as signal processing, machine learning, cryptography, and high-performance computing.

3. Hardware/Software Co-design:

- FPGAs enable hardware/software co-design, where critical parts of an application are offloaded to hardware accelerators implemented in the FPGA.

- This approach can improve system performance, reduce power consumption, and enable real-time processing of data.

4. Reusability:

- FPGAs promote reusability of intellectual property (IP) cores, allowing designers to reuse existing hardware components and designs across different projects and applications.

Applications of FPGAs:

1. Embedded Systems:

- FPGAs are used in embedded systems for tasks such as interfacing with sensors and peripherals, implementing real-time control algorithms, and accelerating data processing.

2. Networking and Communications:

- FPGAs are deployed in networking equipment (routers, switches, network interface cards) to handle packet processing, traffic management, and protocol offloading.

3. Signal Processing:

- FPGAs are employed in digital signal processing (DSP) applications, including audio/video processing, image processing, telecommunications, and radar/sonar systems.

4. High-Performance Computing (HPC):

- FPGAs are integrated into high-performance computing systems to accelerate computationally intensive workloads, such as scientific simulations, cryptography, and data analytics.

5. Prototyping and Emulation:

- FPGAs serve as prototyping platforms for ASIC (Application-Specific Integrated Circuit) development, enabling designers to validate and refine hardware designs before manufacturing.

Limitations of FPGAs:

1. Complexity and Learning Curve:

- Designing with FPGAs requires specialized knowledge of hardware description languages (HDLs) such as Verilog and VHDL, as well as understanding FPGA architecture and design constraints.

2. Resource Limitations:

- FPGAs have finite resources (logic elements, memory blocks, and I/O pins) that may limit the complexity and size of designs that can be implemented.

3. Cost:

- FPGAs can be more expensive than off-the-shelf microcontrollers or ASICs for high-volume production, especially for large and complex designs.

4. Power Consumption:

- While FPGAs offer flexibility and performance advantages, they may consume more power compared to fixed-function ASICs or microcontrollers for certain applications.

In summary, FPGAs are powerful and versatile devices that offer unmatched flexibility and customization for digital circuit design. They are widely used in a variety of applications, from embedded systems and networking to signal processing and high-performance computing, where performance, flexibility, and rapid prototyping are critical requirements.

CHAPTER 3

INSTALLATION

3.1 TERMUX

Installing Apps

1. Install F-Droid.
2. Open f-droid on your mobile and install the following apps:
    - Termux
    - Termux:API

Setting up Termux

1. Give termux access to your user directory in Android.

"termux-setup-storage"

2. Upgrade packages (any **one** command)

"pkg upg"

"apt update && apt upgrade"

3. Install mandatory packages (any **one** command)

"apt install build-essential openssh curl git wget subversion silversearcher-ag imagemagick proot proot-distro python bsdtar mutt nmap neovim"

"pkg in build-essential openssh curl git wget subversion silversearcher-ag imagemagick proot proot-distro python bsdtar mutt nmap neovim"

Installing and Setting up Ubuntu on Termux

1. Install ubuntu

"proot-distro install debian"

"proot-distro login debian"

Inside proot-distro

"apt update && apt upgrade"

"apt install apt-utils build-essential cmake neovim git wget subversion imagemagick nano ranger python3-venv"

2. Installing python3

"apt install python3-pip python3-numpy python3-scipy python3-matplotlib python3-mpmath python3-sympy python3-cvxopt"

3. Installing neovim and ranger

“apt install neovim ranger libxtst-dev libx11-dev python3-pynvim”

4. Installing LaTeX

“apt install texlive-full gnumeric”

5. Setup neovim and ranger

#Installing neovim

#Debian

sudo apt install neovim ranger libxtst-dev libx11-dev python3-pynvim

pip3 install ueberzug

#Installing vim-plug

curl -fLo ~/.local/share/nvim/site/autoload/plug.vim --create-dirs
https://raw.githubusercontent.com/junegunn/vim-plug/master/plug.vim

curl -fLo ~/.config/nvim/init.vim --create-dirs
https://raw.githubusercontent.com/gadepall/termux/main/neovim/init.vim

#Enter plugin

nvim ~/.config/nvim/init.vim

:PlugInstall

:qa!

#Beginners may ignore the following

#For installing a plugin

#call plug#begin('~/.config/nvim/plugged')

#Plug 'lervag/vimtex'

#call plug#end()

nvim

:UpdateRemotePlugins

:q!

#To remove a plugin

#call plug#begin('~/.config/nvim/plugged')

#Plug 'lervag/vimtex'

#call plug#end()

:w

:!source ~/.config/nvim/init.vim

:PlugClean

Enter y

#Arch

sudo pacman -Sy ranger python-pynvim ueberzug

#All other instructions remain the same

6. Usage Tips:

Command	Mode	Action
nvim	Terminal	Open nvim editor
nvim filename	Command	Opens file
nvim filename1 filename2	Command	Opens multilple files
:tabs	Command	lists all open tabs
gt	Command	navigate tabs
2gt	Command	Go to tab number 2
i	Edit	Edit Mode
o	Edit	Edit mode in the next line
a	Edit	Edit Mode, next character
Esc	Command	Command Mode
j	Command	Got to next line
k	Command	Got to previous line
l	Command	Go to next character in the same line
h	Command	Go to previous character in the same line
p	Command	Paste
v	Visual	Select
Shift+v	Visual	Select Lines
Ctrl+v,j,Shift+I,%,Esc	Visual	Comment current line and next line in latex file
Ctrl+v,j,x	Visual	Uncomment current line and next line
dd	Command	Delete current line
3dk	Command	Delete 3 lines before current line
di{	Command	Delete all within {}
yy	Command	copy current line
3yy	Command	copy the next 3 lines
w	Command	Go to next word
b	Command	Go to previous word
dw	Command	Delete current word
0	Command	Go to beginning of the line
:1	Command	Go to the first line in the file
:10	Command	Go to line 10 in the file
ctrl+^	Command	Go to beginning of the line

$	Command	Go to end of line
g,_	Command	Go to end of line
Shift+g	Command	Go to end of file
gt	Command	Go to next file
3gt	Command	Go to 3rd file in tab
Ctrl+W,c	Command	Close current file
:	Command	Use command mode
:w	Command	Save file
:/	Command	Search for string
:tabnew filename	Command	Open file in a new tab
:3tabdo wq	Command	save and close 3rd file
:wq	Command	save and close file
:q!	Command	Close file without saving
:.,$tabdo wq	Command	Close all files, starting from the current file
:1,3tabdo wq	Command	Close files 1 and 3
:s@find@replace@g	Command	find and replace all words in current line
:%s@find@replace@gc	Command	find and replace all words in the current file, but with y/n condition
:.,$s@find@replace@gc	Command	find and replace from the current line to the last line
!ls	Command	List all files/folders in current directory
q:	Command	Allows you to use vim commands in the command mode
Ctrl+R,%	Insert	Prints the path of currently open file in edit mode
:u	Command	Undo
:buffers	Command	Lists all buffers
:sbuffer 1	Command	Opens buffer number 1 in a split window
Ctrl>+<w		
:bufdo e	Command	**Reload all buffers in Vim**
:tab sb 2	Command	Open buffer 2 file in new tab
"a7yy	Command	Yank next seven lines into buffer a
"ap	Command	Paste after cursor

3.2 PlatformIO

apt update && apt upgrade

apt install apt-utils build-essential cmake neovim

apt install git wget subversion imagemagick nano

apt install avra avrdude gcc-avr avr-libc

#------------------End Installing ubuntu on termux----------------------------

#------------------ Installing python3 on termuxubuntu----------------------------

apt install python3-pip python3-numpy python3 -scipy python3-matplotlib python3-mpmath
python3-sympy python3-cvxopt

#------------------ End installing python3 on termuxubuntu----------------------------

#---------- Installing platformio on termuxubuntu----------

pip3 install platformio

#---------- End installing python3 on termuxubuntu----------

2. Execute the following on ubuntu

```
cd /sdcard/Download
svn co https://github.com/gadepall/fwc-1/trunk/
      ide/piosetup/codes
cd codes pio
run
```

3. Connect your arduino to the laptop/rpi and type
   "pio run -t nobuild -t upload"

4. The LED beside pin 13 will start blinking.

Arduino Droid

1. Install ArduinoDroid from apkpure
2. Open ArduinoDroid and grant all permissions
3. Connect the Arduino to your phone via USBOTG
4. For flashing the bin files, in ArduinoDroid,
   Actions->Upload->Upload Precompiled

then go to your working directory and select

.pio/build/uno/firmware.hex

for uploading .hex file to the Arduino Uno.

5. The LED beside pin 13 will start blinking.

3.3 ESP32

#On termux

apt update && apt upgrade

apt install python3-pip

pip3 install platformio

#Download sample directory for esp32 and arduino

svn co https://github.com/gadepall/termux/trunk/pio/Projects/hi

pio lib --global install "stempedia/DabbleESP32"

#Install xtensa and espressif32

cd Projects/hi

pio run

#This will also build firmware.bin in

.pio/build/esp32doit-devkit-v1/firmware.bin

#comment the following line like this if you are using your laptop

;platform_packages = toolchain-xtensa32@https://github.com/esphome/esphome-docker-base/releases/download/v1.4.0/toolchain-xtensa32.tar.gz

#On your rpi

pip3 install platformio

mkdir -p ~/hi/.pio/build/esp32doit-devkit-v1/

cd hi

wget https://raw.githubusercontent.com/gadepall/termux/main/pio/Projects/hi/platformio.ini

nano platformio.ini

#comment the following line like this if you are using rpi3

;platform_packages = toolchain-xtensa32@https://github.com/esphome/esphome-docker-base/releases/download/v1.4.0/toolchain-xtensa32.tar.gz

#From your phone

scp .pio/build/esp32doit-devkit-v1/firmware.bin pi@192.168.50.252:./hi/.pio/build/esp32doit-devkit-v1/firmware.bin

#On your rpi

cd ~/hi

pio run -t nobuild -t upload

----------End ESP32 installation ----------

3.4 ARM

proot-distro login debian

apt update

apt upgrade

#Install ssh-server

apt install openssh-server

Install arm toolchain and hardware tools

apt install build-essential libssl-dev libffi-dev python3-dev bison flex git tcl-dev tcl-tclreadline libreadline-dev autoconf libtool make automake texinfo pkg-config libusb-1.0-0 libusb-1.0-0-dev gcc-arm-none-eabi libnewlib-arm-none-eabi telnet python3 apt-utils libxslt-dev python3-lxml python3-simplejson cmake curl python3-pip

#ARM Hello world

#Test the GNU RM GCC Embedded toolchain

arm-none-eabi-gcc --version

#Download the pygmy-sdk

cd /data/data/com.termux/files/home/

git clone --recursive https://github.com/optimuslogic/pygmy-dev

#Download the helloworld program

cd /data/data/com.termux/files/home/

mkdir arm-examples

cd arm-examples

svn co https://github.com/gadepall/vaman/trunk/arm/codes/setup/blink

cd blink/GCC_Project

make -j4

#blink.bin should be generated in ouput/bin/ directory

#Transfer .bin file to RPI

scp output/bin/blink.bin pi@192.168.0.114:

#Suitably modify the ip address above

#On RPI execute the following

#If Tinyfpga is not installed

git clone --recursive https://github.com/QuickLogic-Corp/TinyFPGA-Programmer-Application.git

sudo apt install python3-pip

sudo pip3 install tinyfpgab pyserial

sudo reboot

Connect the raspberry pi to Vaman through USB.

On the left of the USB port, an LED and a button can be seen. Another button is visible on the right of the USB port.

Press the right button and immediately press the left button. The green LED starts blinking. The Vaman is now in download mode.

mode.

#This will flash the .bin file to the pygmy

sudo python3 /home/pi/TinyFPGA-Programmer-Application/tinyfpga-programmer-gui.py --port /dev/ttyACM0 --m4app blink.bin --mode m4

#On termuxdebian, the source c file is located at

blink/src/main.c

#You may make changes here to modify the colour of the led etc..

#End ARM testing

#In case the default arm-gcc does not work

##For RPI4 in 64 -bit mode (debian 20.04):

##OR for Termux on Android (debian 20.04 proot)#Download and unzip:

#cd $INSTALL_DIR

#wget https://developer.arm.com/-/media/Files/downloads/gnu-rm/9-2020q2/gcc-arm-none-eabi-9-2020-q2-update-aarch64-linux.tar.bz2

#tar -xvjf gcc-arm-none-eabi-9-2020-q2-update-aarch64-linux.tar.bz2

#export PATH=$INSTALL_DIR/gcc-arm-none-eabi-9-2020-q2-update/bin:$PATH

3.5 FPGA

This contains steps to install fpga and run successfully

Steps to follow:

1) Install the zip provided i.e **fpga.zip** onto **Desktop**

2) now open terminal and go to the fpga folder

3) In here **/Desktop/fpga** type: **chmod +x setup.sh** --> This command is particularly useful for making scripts and programs executable. In this way, users can run them without typing their full path or using a dot slash (./) before their name.

4) **sudo bash setup.sh**

5) After running the above command, Go to: **file Manager --> Other Locations --> Computer --> root** . Copy the files **Symbiflow** and **Pygmy-dev** paste them in **"/desktop/fpga"**

6) Now download **arch.tar.gz** using this below link and also place it in **/desktop/fpga**:

<pre>https://iith-my.sharepoint.com/:u:/g/personal/gadepall_ee_iith_ac_in/Ebot5QHEYXBAo-7n4hnvJu0B8vMrTIdj_COHJC2cmDY1ww?e=bqDxHI</pre>

7) Make sure that **/desktop/fpga** contains **pygmy-dev**, **setup.sh** and **symbiflow** and zip file **arch.tar.gz**

8) Next:

At Home:

<pre>

sudo apt update -y

sudo apt upgrade -y

sudo apt install openssh-server sshpass build-essential libssl-dev libffi-dev python3-dev bison flex git tcl-dev tcl tcl-tclreadline libreadline-dev autoconf libtool make automake texinfo pkg-config libusb-1.0-0 libusb-1.0-0-dev gcc-arm-none-eabi libnewlib-arm-none-eabi telnet python3 apt-utils libxslt-dev cmake curl python3-pip python3-venv -y

pip3 install gdown lxml simplejson

sudo apt install openssh-server sshpass

sudo apt install build-essential libssl-dev libffi-dev python3-dev bison flex git tcl-dev tcl tcl-tclreadline libreadline-dev autoconf libtool make automake texinfo pkg-config libusb-1.0-0 libusb-1.0-0-dev gcc-arm-none-eabi libnewlib-arm-none-eabi telnet python3 apt-utils libxslt-dev python3-lxml python3-simplejson cmake curl python3-setuptools python3-pip

cp arch.tar.gz /home/nithin/Desktop/fpga/arch.tar.gz

export INSTALL_DIR=/home/"Username"/Desktop/fpga/symbiflow

tar -C $INSTALL_DIR -xvf /home/nithin/Desktop/fpga/arch.tar.gz

export PATH="$INSTALL_DIR/quicklogic-arch-defs/bin:$INSTALL_DIR/quicklogic-arch-defs/bin/python3:$PATH"

cd /home/nithin/Desktop/fpga/pygmy-dev/tools/quicklogic-fasm

nvim requirements.txt

Replace contents with these lines:

-e git+https://github.com/symbiflow/fasm.git#egg=fasm

-e git+https://github.com/antmicro/quicklogic-fasm-utils.git#egg=fasm-utils

python3 -m venv venv

source venv/bin/activate

pip3 install -r requirements.txt

sudo python3 setup.py install

cd /home/nithin/Desktop/fpga/pygmy-dev/tools/quicklogic-yosys

make config-gcc

make -j4 install PREFIX=$INSTALL_DIR

cd /home/nithin/Desktop/fpga/pygmy-dev/tools/yosys-symbiflow-plugins

export PATH=$INSTALL_DIR/bin:$PATH

make -j4 install

cd /home/nithin/Desktop/fpga/pygmy-dev/tools/vtr-verilog-to-routing

make -j4

nvim ~/.bashrc

#paste the following 3 lines at the end of the file

export INSTALL_DIR=/home/nithin/Desktop/fpga/symbiflow

export PATH="$INSTALL_DIR/quicklogic-arch-defs/bin:$INSTALL_DIR/quicklogic-arch-defs/bin/python3:$PATH"

export PATH=/home/nithin/Desktop/fpga/symbiflow/bin:$PATH

source ~/.bashrc

vpr -h

yosys -h

qlfasm -h

ql_symbiflow -h

cd $INSTALL_DIR/quicklogic-arch-defs/tests/counter_16bit

ls

ql_symbiflow -compile -d ql-eos-s3 -P pd64 -v counter_16bit.v -t top -p chandalar.pcf -dump binary

pip3 install gdown lxml simplejson

ql_symbiflow -compile -d ql-eos-s3 -P pd64 -v counter_16bit.v -t top -p chandalar.pcf -dump binary

ls

cd /home/nithin/Desktop/fpga

ls

mkdir fpga-examples

cd fpga-examples

svn co https://github.com/gadepall/vaman/trunk/fpga/setup/codes/blink

cd blink

ls

Command to compile code:

ql_symbiflow -compile -src /home/nithin/Desktop/fpga/fpga-examples/blink -d ql-eos-s3 -P PU64 -v helloworldfpga.v -t helloworldfpga -p quickfeather.pcf -dump binary

ls

nvim helloworldfpga.v --> This file contains the code

nvim quickfeather.pcf --> This file contains pin configurations

Make sure to compile the file after modifying .v or .pcf file

Command to upload code to vaman:

#Make sure that vaman is in upload mode(i.e Green Light)

After Compiling the code make sure to copy the bin file and paste it in home and perform the upload command

sudo python3 /home/nithin/TinyFPGA-Programmer-Application/tinyfpga-programmer-gui.py --port /dev/ttyACM0 --appfpga /home/nithin/helloworldfpga.bin --mode fpga

</pre>

If Errors get Popped Up:

#Install fasm

cd /home/nithin/Desktop/fpga/pygmy-dev/tools/quicklogic-fasm

nvim requirements.txt

#modify the lines so that they look like

-e git+https://github.com/symbiflow/fasm.git#egg=fasm

-e git+https://github.com/antmicro/quicklogic-fasm-utils.git#egg=fasm-utils

sudo pip3 install -r requirements.txt

sudo python3 setup.py install

last 2 cmds try with and without sudo

#add path to .bashrc

nvim ~/.bashrc

#paste the following 3 lines at the end of the file

export INSTALL_DIR=/home/nithin/Desktop/fpga/symbiflow

export PATH="$INSTALL_DIR/quicklogic-arch-defs/bin:$INSTALL_DIR/quicklogic-arch-defs/bin/python3:$PATH"

export PATH=/home/nithin/Desktop/fpga/symbiflow/bin:$PAT

#Ctrl+X save and exit. reboot for the settings to be updated.

#reboot make sure above 3 lines are in bach file and after try rebooting the system.

3.6 Ubuntu

1. Backup your data: Before you start, it's crucial to back up all your important data in case anything goes wrong during the installation process.

2. Create a bootable Ubuntu USB drive:

- Download the Ubuntu ISO file from the official Ubuntu website (https://ubuntu.com/download).

- Use a tool like Rufus (for Windows) or balenaEtcher (for Windows, macOS, or Linux) to create a bootable USB drive with the Ubuntu ISO file.

- Insert the USB drive into a USB port on your computer.

3. Shrink Windows partition:

- Press `Win + X` and choose "Disk Management" (or search for "Disk Management" in the Start menu).

- Right-click on the partition you want to shrink (usually the C: drive), and select "Shrink Volume".

- Enter the amount of space you want to shrink. Make sure you leave enough space for Ubuntu (at least 20 GB is recommended).

- Click "Shrink" to resize the partition.

4. Disable Fast Startup (Windows 8 and 10):

- Open Control Panel and go to "Power Options".

- Click on "Choose what the power buttons do".

- Click on "Change settings that are currently unavailable".

- Scroll down and uncheck "Turn on fast startup (recommended)".

- Click "Save changes".

5. Disable Secure Boot (if necessary):

- Restart your computer and enter BIOS/UEFI settings (usually by pressing F2, F12, ESC, or DEL during boot).

- Navigate to the "Security" or "Boot" tab.

- Disable Secure Boot.

- Save changes and exit.

6. Boot from the Ubuntu USB drive:

- Restart your computer.

- Enter BIOS/UEFI settings (usually by pressing F2, F12, ESC, or DEL during boot).

- Change the boot order to prioritize booting from the USB drive.

- Save changes and exit.

7. Install Ubuntu:

- When the Ubuntu installer boots, select "Install Ubuntu" from the menu.

- Choose your language and click "Continue".

- Select "Install Ubuntu alongside Windows Boot Manager" and click "Install Now".

- Follow the on-screen instructions to select your timezone, keyboard layout, and create a user account.

- When prompted, choose the partition you created earlier for Ubuntu installation.

- Click "Install Now" and then "Continue" to start the installation process.

- Once the installation is complete, restart your computer.

8. Grub Bootloader:

- After restarting, you'll see the GRUB bootloader menu, which allows you to choose between Ubuntu and Windows.

- Select the operating system you want to boot into using the arrow keys and press Enter.

9. Finalize Installation:

- Once Ubuntu loads, complete the initial setup by following the on-screen instructions.

- Make sure everything works properly in both Ubuntu and Windows.

CHAPTER 4

METHODOLOGY

4.1 Latex Tasks

1. Open Termux and login to Debian.
2. Navigate to the folder where the files must be saved.
3. Enter the command **"nvim <file_name>.tex"**.
4. Write the latex code by clicking **"ALT+i"** after writing the code click **"ALT+ESC"**.
5. To execute the code file enter the command **"pdflatex <file_name>.tex"**.
6. The pdf file will be successfully generated.

4.2 Digital Design Tasks using C++ Codes

1. Open Termux and login to Debian.
2. Navigate to the code file location using **RANGER**.
3. In order to navigate using RANGER enter the command **"ranger"**.
4. After navigating to the location click **"SHIFT+S"**.
5. Enter the command **"piorun"** to execute the code file.
6. If you want to edit the code files then use nvim editor.
7. After execution open **"ArduinoDroid"** in mobile.
8. Connect the mobile with the Arduino board using OTG cable.
9. In the ArduinoDroid app click on three dots.
10. Click on actions.
11. Click on upload pre-compiled.
12. Navigate to the bin file.
13. Select the bin file and upload it to the board.

4.3 Assembly Task

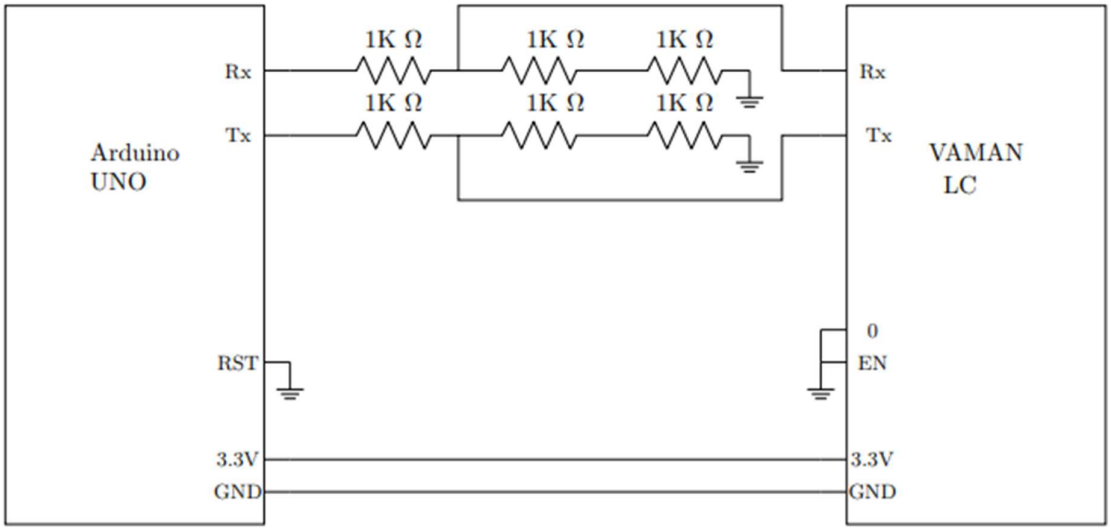
1. Open termux and login to Debian.
2. Navigate to the code file folder using **RANGER**.
3. To open the new assembly code use the command **"<file_name>.asm"**.
4. Write or edit the code file and save it.
5. To execute the code use the command **"avra <file_name>.asm"**.
6. To upload the compiled the output file use the ArduinoDroid steps as said above.

4.4 AVR – GCC Task

1. Open termux and login to Debian.
2. Navigate to the code file location using **RANGER**.
3. Write or edit the code file using nvim editor.
4. Open the file location in **RANGER** and click **"SHIFT+S"**.
5. Enter the command **"make"** to execute the code file.
6. Upload the output file using ArduinDroid.

4.5 ESP32 Task

1. Open termux and login to Debian.
2. Navigate to the esp32 code file location.
3. Edit the code file.
4. Save it and execute the file using the command **“pio run”**.
5. After successful compilation make the below mentioned connections with Arduino uno board and Vaman board as shown in the below picture.



6. Then flash the output file to it and remove the connections after successful output file flashing.
7. While flashing the output file remove the “en” pin when the upload gets started and remove the “0” pin connection after uploading the file.
8. Then try to connect to the hotspot of the mobile with the vaman.
9. You must be able to see the connected esp device in the connected devices list of the hotspot.
10. Use the below command to upload the code file to the vaman wirelessly:
    “pio run -t nobuild -t upload”.

4.4 ARM Task

1. Open Ubuntu.
2. Navigate to the ARM code file using **RANGER**.
3. Edit or write the code file using the nvim editor and save it.
4. Navigate to the file location using RANGER and click **“SHIFT+S”**.
5. To execute the file enter the command **“make”**.
6. After successful execution upload the output file using the ArduinoDroid.
   Command to flash the output file:
   “sudo python3 /home/pi/TinyFPGA-Programmer-Application/tinyfpga-programmer-gui.py --port /dev/ttyACM0 --m4app blink.bin --mode m4”.

4.5 FPGA Task

1. Open ubuntu terminal.
2. Open the file location in the ubuntu terminal using the “cd” command.
3. Edit the .v code file with the code logic.
4. Update the .pcf file with the circuit connections.
5. Use the command to execute the code and connection file.
   “ql_symbiflow -compile -src /home/nithin/Desktop/fpga/fpga-examples/blink -d ql-eos-s3 -P PU64 -v helloworldfpga.v -t helloworldfpga -p quickfeather.pcf -dump binary”.
6. Flash the output file to the vaman board by connecting it to the PC.
   Command to upload the output file:
   “sudo python3 /home/nithin/TinyFPGA-Programmer-Application/tinyfpga-programmer-gui.py --port /dev/ttyACM0 --appfpga /home/nithin/helloworldfpga.bin --mode fpga”.

CHAPTER 5

CODES & OUTPUTS

5.1 Latex Codes & Outputs

5.1.1 Integration code:

```
\begin{enumerate}
\item Evaluate:
   \begin{align}
      \int^{\frac{\pi}{2}}_{_\frac{-\pi}{2}}x \cos^2x dx
   \end{align}
      \item Find the integrating factor of the differential equation
\begin{align}
   x \frac{dy}{dx} = 2x^2 + y
\end{align}
\item Find:
\begin{align}
   \int \frac{\tan^3 x}{\cos^3 x} dx
\end{align}
\item Solve the following differential equation:
\begin{align}
   \brak{1+e^\frac{y}{x}}dy + e^\frac{y}{x}\brak{1-\frac{y}{x}} dx = 0 \brak{x \neq 0}
\end{align}
\item Evaluate:
\begin{align}
       \int^{\frac{\pi}{2}}_{0} \sin 2x \tan^{-1} (\sin x) dx
\end{align}
\item Using integration, find the area lying above x-axis and included between the circle $x^2 + y^2 = 8x$ and inside the parabola $y^2 = 4x$.
\item Using the method of integration, find the area of the triangle ABC, coordinates of whose vertices are $A\brak{2,0}$, $B\brak{4,5}$ and $C\brak{6,3}$.
\end{enumerate}
```

5.1.2 Geometry Code:

```
\begin{enumerate}
\item The radius of a sphere (in cm) whose volume is $12\pi cm^3$, is
\begin{enumerate}
\item $3$
\item $3 \sqrt{3}$
\item $3^\frac{2}{3}$
\item $3^\frac{1}{3}$
\end{enumerate}
\item In \figref{fig:Fig-4.png}, the angle of elevation of the top of a tower from a point $C$ on the ground, which is $30m$ away from the foot of the tower, is $30\degree$. Find the height of the tower.
\begin{figure}[H]
\centering
\includegraphics[width=\columnwidth]{Documents/Projects/Figures/Fig-4.png}
\caption{}
\label{fig:Fig-4.png}
\end{figure}
\item A cone and a cylinder have the same radii but the height of the cone is $3$ times that of the cylinder. Find the ratio of their volumes.
```

\item In \figref{fig:Fig-8.png}, $ABCD$ is a parallelogram. A semicircle with centre O and the diameter AB has been drawn and it passes through D. If $AB=12cm$ and $OD \perp AB$, then find the area of the shaded region. (Use $\pi=3.14$)
\begin{figure}[H]
\centering
\includegraphics[width=\columnwidth]{Documents/Projects/Figures/Fig-8.png}
\caption{}
\label{fig:Fig-8.png}
\end{figure}
\item A statue $1.6m$ tall, stands on the top of a pedestal. From a point on the ground, the angle of elevation of the top of the statue is $60\degree$ and from the same point the angle of elevation of the top of the pedestal is $45\degree$. Find the height of the pedestal.(Use $\sqrt{3}=1.73$)
\item In a cylindrical vessel of radius $10 cm$, containing some water, 9000 small spherical balls are dropped which are completely immersed in water which raises the water level. If each spherical ball is of radius $0.5 cm$, then find the rise in the level of water in the vessel.
\item If a line is drawn parallel to one side of a triangle to intersect the other two sides at distinct points, prove that the other two sides are divided in the same ratio.
\item If $\tan^{-1}\brak{\frac{y}{x}}=\log\sqrt{x^2+y^2}$, prove that $\frac{dy}{dx}=\frac{x+y}{x-y}$.
\item If $y=e^{a \cos^{-1}x}, -1<x<1$, then show that
\begin{align}
 \brak{1-x^2}\frac{d^2y}{dx^2}-x\frac{dy}{dx}-a^2y=0
\end{align}
\end{enumerate}

GEOMETRY - CBSE

TIRUMALA SAI NITHIN

Dec 2023

1. The radius of a sphere (in cm) whose volume is $12\pi cm^3$, is

   (a) 3

   (b) $3\sqrt{3}$

   (c) $3^{\frac{1}{3}}$

   (d) $3^{\frac{1}{2}}$

2. In Figure **??**, the angle of elevation of the top of a tower from a point C on the ground, which is $30m$ away from the foot of the tower, is $30°$. Find the height of the tower.

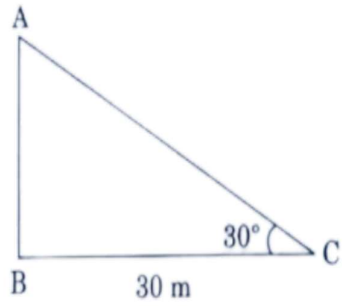


Figure 1

3. A cone and a cylinder have the same radii but the height of the cone is 3 times that of the cylinder. Find the ratio of their volumes.

4. In Figure **??**, $ABCD$ is a parallelogram. A semicircle with centre O and the diameter AB has been drawn and it passes through D. If $AB = 12cm$ and $OD \perp AB$, then find the area of the shaded region. (Use $\pi = 3.14$)

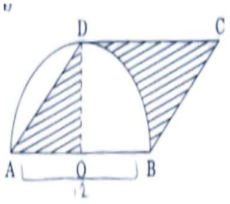


Figure 2

5. A statue 1.6*m* tall, stands on the top of a pedestal. From a point on the ground, the angle of elevation of the top of the statue is 60° and from the same point the angle of elevation of the top of the pedestal is 45°. Find the height of the pedestal.(Use $\sqrt{3} = 1.73$)

6. In a cylindrical vessel of radius 10*cm*, containing some water, 9000 small spherical balls are dropped which are completely immersed in water which raises the water level. If each spherical ball is of radius 0.5*cm*, then find the rise in the level of water in the vessel.

7. If a line is drawn parallel to one side of a triangle to intersect the other two sides at distinct points, prove that the other two sides are divided in the same ratio.

8. If $\tan^{-1}\left(\frac{y}{x}\right) = \log\sqrt{x^2+y^2}$, prove that $\frac{dy}{dx} = \frac{x+y}{x-y}$.

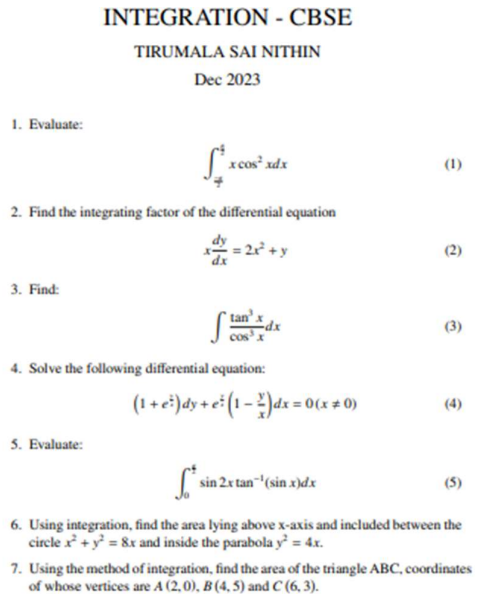
9. If $y = e^{a\cos^{-1}x}$, $-1 < x < 1$, then show that

$$\left(1-x^2\right)\frac{d^2y}{dx^2} - x\frac{dy}{dx} - a^2y = 0 \qquad (1)$$

Geometry Output Figures

5.1.3 GATE Code:

\item The digital circuit shown in \figref{fig:GATEIN202236.png}
\begin{figure}[H]
 \centering
 \includegraphics[width=\columnwidth]{ide/7474/figs/GATEIN202236.png}
 \caption{}
 \label{fig:GATEIN202236.png}
\end{figure}
 \begin{enumerate}
 \item is a divide-by-5 counter
 \item is a divide-by-7 counter
 \item is a divide-by-8 counter
 \item does not function as a counter due to disjoint cycles of states
\end{enumerate}
\hfill(GATE-IN-2022)



INTEGRATION - CBSE

TIRUMALA SAI NITHIN

Dec 2023

1. Evaluate:

$$\int_{\frac{\pi}{6}}^{\frac{\pi}{4}} x\cos^2 x dx \quad (1)$$

2. Find the integrating factor of the differential equation

$$x\frac{dy}{dx} = 2x^2 + y \quad (2)$$

3. Find:

$$\int \frac{\tan^3 x}{\cos^3 x} dx \quad (3)$$

4. Solve the following differential equation:

$$\left(1 + e^{\frac{x}{y}}\right)dy + e^{\frac{x}{y}}\left(1 - \frac{y}{x}\right)dx = 0 \, (x \neq 0) \quad (4)$$

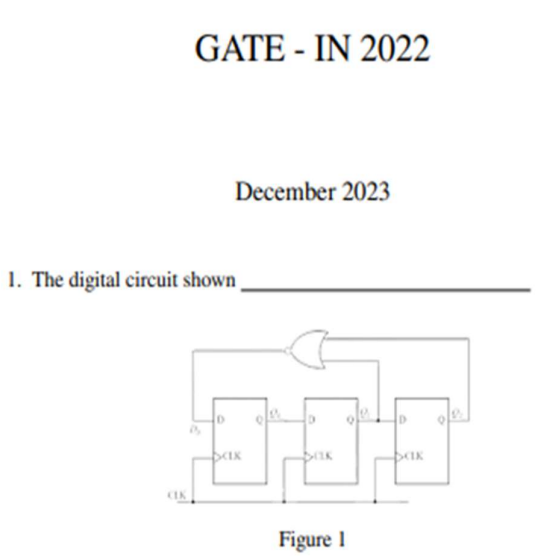
5. Evaluate:

$$\int_0^{\frac{\pi}{4}} \sin 2x \tan^{-1}(\sin x) dx \quad (5)$$

6. Using integration, find the area lying above x-axis and included between the circle $x^2 + y^2 = 8x$ and inside the parabola $y^2 = 4x$.

7. Using the method of integration, find the area of the triangle ABC, coordinates of whose vertices are $A(2, 0)$, $B(4, 5)$ and $C(6, 3)$.

Integration Output



GATE - IN 2022

December 2023

1. The digital circuit shown _____

Figure 1

GATE Output

5.2 Digital Design Codes & Outputs – Arduino Uno

5.2.1 Seven Segment

Code:

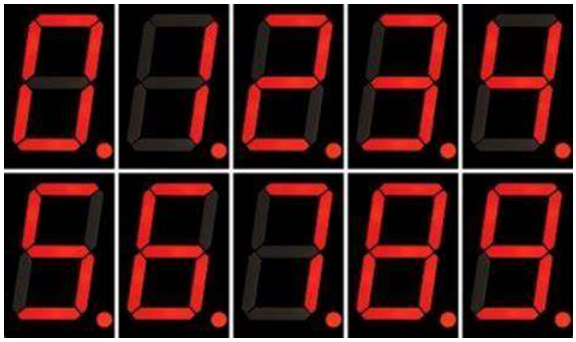
```
#define aPin 7  //
#define bPin 8  //    ___
#define cPin 2  //    | A |
#define dPin 3  //  F |___| B
#define ePin 4  //    | G |
#define fPin 6  //  E |___| C
#define gPin 5  //      D    O dot

// Pin configuration
#define PIN_BUTTON A0
#define PIN_BUZZER 10

const byte PIN_CHAOS = A5; // Unconnected analog pin used to initialize RNG

// Other configuration
const unsigned int BEEP_FREQUENCY = 3000;

int On = 1; //<On=0; for Common anode><On=1; for Common cathode>
int Off;

void setup() {
 randomSeed(analogRead(PIN_CHAOS));

 pinMode(aPin, OUTPUT);
 pinMode(bPin, OUTPUT);
 pinMode(cPin, OUTPUT);
```

```
  pinMode(dPin, OUTPUT);
  pinMode(ePin, OUTPUT);
  pinMode(fPin, OUTPUT);
  pinMode(gPin, OUTPUT);

  pinMode(PIN_BUTTON, INPUT_PULLUP);    // On button pin as input with pullup
  pinMode(PIN_BUZZER, OUTPUT);      // On buzzer pin as output

  // Indicate that the system is ready
  for (int i = 0; i <= 9; i++) {
    showNumber(i);
    tone(PIN_BUZZER, BEEP_FREQUENCY, 100);
    delay(300);
  }
  tone(PIN_BUZZER, BEEP_FREQUENCY, 250);  // Beep when done
  delay(1000);              // Wait for 1 second
}

void loop() {
  // Wait for the button to be pressed, then run the test routine
  int buttonState = digitalRead(PIN_BUTTON);
  if (buttonState == LOW) {
    rollTheDice(10, 100);         // Show the rolling animation with a delay of 100 milliseconds
    rollTheDice(5, 200);
    rollTheDice(3, 300);
    rollTheDice(1, 100);
    tone(PIN_BUZZER, BEEP_FREQUENCY, 250);  // Beep when done
  }
}

void rollTheDice(int count, int delayLength) {
  for (int i = 0; i < count; i++) {
    int number = random(1, 7);      // Get a random number from 1 to 6
    tone(PIN_BUZZER, BEEP_FREQUENCY, 5);  // Beep very shortly ("click")
    showNumber(number);          // Show the number
    delay(delayLength);          // Wait
  }
}

void showNumber(int x){

  if (On == 1) {
    Off = 0;
  } else {
    Off = 1;
  }

  switch(x) {
    case 1: one();   break;
    case 2: two();   break;
    case 3: three(); break;
    case 4: four();  break;
    case 5: five();  break;
    case 6: six();   break;
    case 7: seven(); break;
    case 8: eight(); break;
    case 9: nine();  break;
    default: zero(); break;
```

```
  }
}

void one() {
  digitalWrite(aPin, On); //
  digitalWrite(bPin, Off);  //     |
  digitalWrite(cPin, Off);  //     |
  digitalWrite(dPin, On); //     |
  digitalWrite(ePin, On); //     |
  digitalWrite(fPin, On);
  digitalWrite(gPin, On);
  delay(500);
}

void two() {
  digitalWrite(aPin, Off);  //  __
  digitalWrite(bPin, Off);  //     |
  digitalWrite(cPin, On); //  __|
  digitalWrite(dPin, Off);  // |
  digitalWrite(ePin, Off);  // |__
  digitalWrite(fPin, On);
  digitalWrite(gPin, Off);
  delay(500);
}

void three() {
  digitalWrite(aPin, Off);  //  __
  digitalWrite(bPin, Off);  //     |
  digitalWrite(cPin, Off);  //  __|
  digitalWrite(dPin, Off);  //     |
  digitalWrite(ePin, On); //  __|
  digitalWrite(fPin, On);
  digitalWrite(gPin, Off);
  delay(500);
}

void four() {
  digitalWrite(aPin, On); //
  digitalWrite(bPin, Off);  // |    |
  digitalWrite(cPin, Off);  // |__|
  digitalWrite(dPin, On); //     |
  digitalWrite(ePin, On); //     |
  digitalWrite(fPin, Off);
  digitalWrite(gPin, Off);
  delay(500);
}

void five() {
  digitalWrite(aPin, Off);  //  __
  digitalWrite(bPin, On); // |
  digitalWrite(cPin, Off);  // |__
  digitalWrite(dPin, Off);  //     |
  digitalWrite(ePin, On); //  __|
  digitalWrite(fPin, Off);
  digitalWrite(gPin, Off);
  delay(500);
}
```

```
void six() {
  digitalWrite(aPin, Off);  //  __
  digitalWrite(bPin, On); // |
  digitalWrite(cPin, Off);  // |__
  digitalWrite(dPin, Off);  // |    |
  digitalWrite(ePin, Off);  // |__|
  digitalWrite(fPin, Off);
  digitalWrite(gPin, Off);
  delay(500);
}

void seven() {
  digitalWrite(aPin, Off);  //  __
  digitalWrite(bPin, Off);  //     |
  digitalWrite(cPin, Off);  //     |
  digitalWrite(dPin, On); //     |
  digitalWrite(ePin, On); //     |
  digitalWrite(fPin, On);
  digitalWrite(gPin, On);
  delay(500);
}

void eight() {
  digitalWrite(aPin, Off); //  __
  digitalWrite(bPin, Off); // |    |
  digitalWrite(cPin, Off); // |__|
  digitalWrite(dPin, Off); // |    |
  digitalWrite(ePin, Off); // |__|
  digitalWrite(fPin, Off);
  digitalWrite(gPin, Off);
  delay(500);
}

void nine() {
  digitalWrite(aPin, Off);  //  __
  digitalWrite(bPin, Off);  // |    |
  digitalWrite(cPin, Off);  // |__|
  digitalWrite(dPin, Off);  //     |
  digitalWrite(ePin, On); //  __|
  digitalWrite(fPin, Off);
  digitalWrite(gPin, Off);
  delay(500);
}

void zero() {
  digitalWrite(aPin, Off); //  __
  digitalWrite(bPin, Off); // |    |
  digitalWrite(cPin, Off); // |    |
  digitalWrite(dPin, Off); // |    |
  digitalWrite(ePin, Off); // |__|
  digitalWrite(fPin, Off);
  digitalWrite(gPin, On);
}
```

Output:



5.2.2 7447

Code :

```
void disp_7447(int D, int C, int B, int A)
{
  digitalWrite(2, A); //LSB
  digitalWrite(3, B);
  digitalWrite(4, C);
  digitalWrite(5, D); //MSB

}
// the setup function runs once when you press reset or power the board
void setup() {
    pinMode(2, OUTPUT);
    pinMode(3, OUTPUT);
    pinMode(4, OUTPUT);
    pinMode(5, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
disp_7447(0,1,0,1);
}
```

Output :

5.2.3 K – Map

Code :

```
//Declaring all variables as integers
int Z=0,Y=0,X=0,W=1;
int D,C,B,A;

//Code released under GNU GPL.  Free to use for anything.
void disp_7447(int D, int C, int B, int A)
```

```
{
  digitalWrite(2, A); //LSB
  digitalWrite(3, B);
  digitalWrite(4, C);
  digitalWrite(5, D); //MSB

}
// the setup function runs once when you press reset or power the board
void setup() {
    pinMode(2, OUTPUT);
    pinMode(3, OUTPUT);
    pinMode(4, OUTPUT);
    pinMode(5, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  A=0;
  B=(W&&!X&&!Y&&!Z) || (!W&&X&&!Y&&!Z) || (W&&!X&&Y&&!Z) || (!W&&X&&Y&&!Z);
  C=(W&&X&&!Y&&!Z) || (!W&&!X&&Y&&!Z) || (W&&!X&&Y&&!Z) || (!W&&X&&Y&&!Z);
  D = (W&&X&&Y&&!Z)||(!W&&!X&&!Y&&Z);

disp_7447(D,C,B,A);
}
```

Output :

W	X	Y	Z		D	C	B	A
---	---	---	---		---	---	---	---
1	0	0	0		0	0	1	0
0	1	0	0		0	1	0	0
1	1	0	0		0	1	0	0
0	0	1	0		0	0	1	0
1	0	1	0		0	0	1	0
0	1	1	0		0	1	0	0
1	1	1	0		0	1	0	0
0	0	0	1		1	0	0	0

For 0 the LED do not glow and for 1 the LED will glow.

5.2.4 7474

Code:

```
#include <Arduino.h>

int W, X, Y, Z;
int D, C, B, A;

void disp_7447(int D, int C, int B, int A) {
digitalWrite(2, A); // LSB
digitalWrite(3, B);
digitalWrite(4, C);
digitalWrite(5, D); // MSB
}

void setup() {
pinMode(2, OUTPUT);
pinMode(3, OUTPUT);
pinMode(4, OUTPUT);
```

```
pinMode(5, OUTPUT);
pinMode(6, INPUT);
pinMode(7, INPUT);
pinMode(8, INPUT);
pinMode(9, INPUT);
pinMode(13, OUTPUT);
}

void loop() {
// Display current state
digitalWrite(13, HIGH);
delay(100);
disp_7447(D, C, B, A);

// Read input switches
W = digitalRead(6);
X = digitalRead(7);
Y = digitalRead(8);
Z = digitalRead(9);

// Calculate next state
A = !W;
B = (!W && !X && Y) || (W && X) || (!W && Z);
C = (!W && Z) || (X && Y) || (W && Y);
D = (!W && !X && !Y && !Z) || (W && Z);

// Update current state with next state
W = A;
X = B;
Y = C;
Z = D;

// Turn off display and delay
digitalWrite(13, LOW);
delay(500);
}
```

Output:

W	X	Y	Z		D	C	B	A
---	---	---	---		---	---	---	---
0	0	0	0		1	0	0	1
1	0	0	0		0	0	0	0
0	1	0	0		0	1	1	1
1	1	0	0		0	1	1	0
0	0	1	0		1	1	0	1
1	0	1	0		0	1	0	0
0	1	1	0		0	1	1	1
1	1	1	0		0	1	1	0
0	0	0	1		1	0	0	1
1	0	0	1		1	1	0	0
0	1	0	1		1	1	1	1
1	1	0	1		0	1	1	0

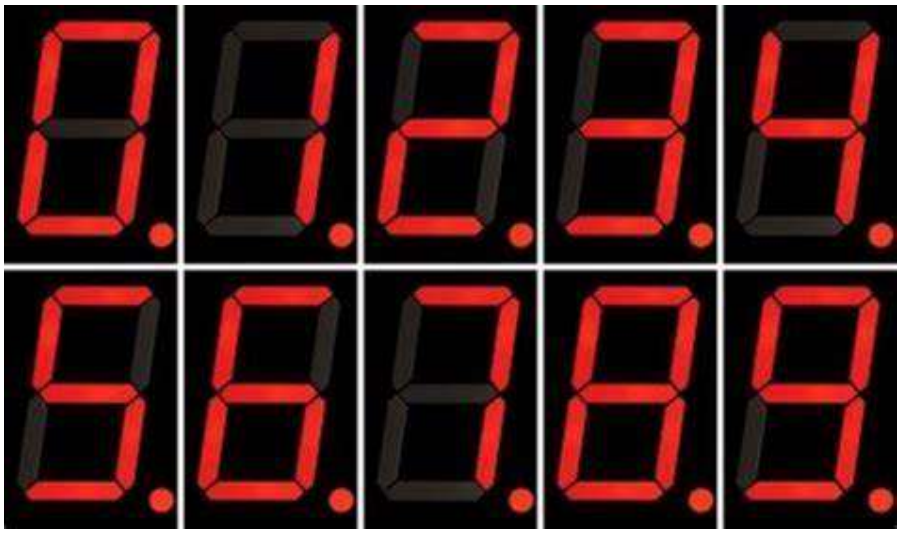
For 0 the LED do not glow and for 1 the LED will glow.

5.2.5 State Machine

Code:

```
#include <Arduino.h>

int W, X, Y, Z;
int D, C, B, A;

void disp_7447(int D, int C, int B, int A) {
 digitalWrite(2, A); // LSB
 digitalWrite(3, B);
 digitalWrite(4, C);
 digitalWrite(5, D); // MSB
}

void setup() {
 pinMode(2, OUTPUT);
 pinMode(3, OUTPUT);
 pinMode(4, OUTPUT);
 pinMode(5, OUTPUT);
 pinMode(6, INPUT);
 pinMode(7, INPUT);
 pinMode(8, INPUT);
 pinMode(9, INPUT);
 pinMode(13, OUTPUT);
}

void loop() {
 // Display current state
 digitalWrite(13, HIGH);
 delay(100);
 disp_7447(D, C, B, A);

 // Read input switches
 W = digitalRead(6);
 X = digitalRead(7);
 Y = digitalRead(8);
 Z = digitalRead(9);

 // Calculate next state
 A = !W;
 B = (!W && !X && Y) || (W && X) || (!W && Z);
 C = (!W && Z) || (X && Y) || (W && Y);
 D = (!W && !X && !Y && !Z) || (W && Z);

 // Update current state with next state
 W = A;
 X = B;
 Y = C;
 Z = D;

 // Turn off display and delay
 digitalWrite(13, LOW);
 delay(500);
}
```

Output:



5.2.6 PlatformIO

Code:

```
#include <Arduino.h>

const int A = A7;  // Assuming A is connected to A7
const int B = B0;  // Assuming B is connected to B0
const int Z = 13;  // Replace 13 with the actual pin number where Z is connected

void setup() {
  // Set the input pins as inputs with pull-up resistors
  pinMode(A, INPUT_PULLUP);
  pinMode(B, INPUT_PULLUP);

  // Set the output pin as output
  pinMode(Z, OUTPUT);
}

void loop() {
  // Read the values of the input pins
  bool a = digitalRead(A);
  bool b = digitalRead(B);

  // Evaluate the Boolean expression for Z
  bool z = (b && a) || !a;

  // Write the value of Z to the output pin
  digitalWrite(Z, z);
}
```

Output:

```
| A | B | C | Z |
| - | - | - | - |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
```

```
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |
```

For 0 LED will not glow, and for 1 LED will glow.

5.2.7 Assembly

Code:

AVR Assembly Code:

```
.include "/sdcard/assembly/m328Pdef.inc"

; Set pin 13 (PB5) as output
SBI DDRB,5

; Infinite loop
start:
   ; Read input from port D
   in r16, PIND
   mov r17, r16

   ; Extract the relevant bits for NAND gate
   andi r16, 0b00000100
   LSR r16
   LSR r16
   ldi r18, 0b00000001
   eor r16, r18

   ; Move the result to r19
   mov r19, r17
   ldi r19, 0b00001000
   LSR r19
   LSR r19
   LSR r19

   ; Perform NAND operation
   and r16, r19

   ; Shift left five times
   LSL r16
   LSL r16
   LSL r16
   LSL r16
   LSL r16

   ; Output the result to pin 13 (PB5)
   out PORTB, r16

   ; Infinite loop
   rjmp start
```

C++ Code:

```
// Define the digital pins for input A and B
const int pinA = 2; // Digital pin 2 for input A
const int pinB = 3; // Digital pin 3 for input B

// Define the digital pin for the LED
const int ledPin = 13; // Built-in LED pin on Arduino Uno

void setup() {
  // Set pin modes
  pinMode(pinA, INPUT);
  pinMode(pinB, INPUT);
```

```
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Read the input values
  int inputA = digitalRead(pinA);
  int inputB = digitalRead(pinB);

  // Compute the output Q based on the truth table
  int outputQ = !(inputA && inputB); // NAND gate operation

  // Turn on/off the LED based on the output Q
  digitalWrite(ledPin, outputQ);

  // Delay for stability
  delay(100);
}
```

ASSEMBLY CODE - 2:

```
; Include necessary files
.include "/sdcard/assembly/m328Pdef.inc"

; Define digital pins for input A and B
.equ pinA, 3 ; Digital pin 3 for input A
.equ pinB, 2 ; Digital pin 2 for input B

; Define digital pin for the LED
.equ ledPin, 5 ; Pin 13 (PB5) for LED

; Set pin 13 (PB5) as output
SBI DDRB, ledPin

; Infinite loop
start:
   ; Read input from ports
   IN r16, PIND
   MOV r17, r16

   ; Extract the relevant bits for NAND gate
   ANDI r16, 0b00000100
   LSR r16
   LSR r16
   LDI r18, 0b00000001
   EOR r16, r18

   ; Move the result to r19
   MOV r19, r17
   LDI r19, 0b00001000
   LSR r19
   LSR r19
   LSR r19

   ; Perform NAND operation
   AND r16, r19

   ; Shift left five times
   LSL r16
   LSL r16
   LSL r16
   LSL r16
   LSL r16

   ; Output the result to pin 13 (PB5)
```

```
OUT PORTB, r16

; Infinite loop
RJMP start
```

Output:

A	B	A AND B	NOT (A AND B) - NAND
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

For 0 LED will not glow, but for 1 LED will glow.

5.2.8 AVR – GCC

Code:

```
#include <avr/io.h>
#include <util/delay.h>

// Define the pin number for the LED
#define LED_PIN 13

// Define the time interval for each state in milliseconds
#define TIME_INTERVAL 1000

void setup() {
  // Set the LED pin as an output
  DDRB |= (1 << DDB5);
}

void loop() {
  // Turn the LED on for state 00
  PORTB |= (1 << PORTB5);
  // Wait for one second
  _delay_ms(TIME_INTERVAL);
  // Turn the LED off for state 10
  PORTB &= ~(1 << PORTB5);
  // Wait for one second
  _delay_ms(TIME_INTERVAL);
  // Turn the LED on for state 01
  PORTB |= (1 << PORTB5);
  // Wait for one second
  _delay_ms(TIME_INTERVAL);
  // Turn the LED off for state 11
  PORTB &= ~(1 << PORTB5);
  // Wait for one second
  _delay_ms(TIME_INTERVAL);
}

int main(void) {
  setup();

  while (1) {
    loop();
  }
```

```
  return 0;
}
```

Output:

State	LED (Binary)	LED (Decimal)
00	1	1
10	0	0
01	1	1
11	0	0

For 0 the LED will not glow, but for 1 the LED will glow.

5.3 Digital Design Codes & Outputs – VAMAN

5.3.1 ESP32

Code:

```
#include <WiFi.h>
#include <WiFiUdp.h>
#include <ArduinoOTA.h>

// Replace with your network credentials
#ifndef STASSID
#define STASSID "Sunitha"
#define STAPSK  "1234567890"
#endif

const char* ssid = STASSID;
const char* password = STAPSK;

void OTAsetup() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.waitForConnectResult() != WL_CONNECTED) {
    delay(5000);
    ESP.restart();
  }
  ArduinoOTA.begin();
}

void OTAloop() {
  ArduinoOTA.handle();
}

void setup() {
  OTAsetup();

  // Custom setup code from code-1
  pinMode(2, OUTPUT);
}

void loop() {
  OTAloop();
  delay(10);  // If no custom loop code, ensure to have a delay in loop
```

```
 // Custom loop code from code-1
 int X = 1;
 int Y = 0;
 int Z = 0;
 int R1 = (X || Y || !Z) && (!X || Y || Z) && (X || Y || !Z) && (!X && !Y || X && Y && Z);
 int R2 = (!X && !Z) || (Y && !Z);
 if (R1 == R2) {
   digitalWrite(2, HIGH);
   delay(1000);
 } else {
   digitalWrite(2, LOW);
   delay(1000);
 }
}
```

Output:

X Y Z F
0 0 0 1
0 0 1 0
0 1 0 1
0 1 1 1
1 0 0 0
1 0 1 0
1 1 0 1
1 1 1 1

For 0 the LED will not glow, but for 1 the LED will glow.

5.3.2 ARM

Code:

```
#include "Fw_global_config.h"
#include <stdio.h>
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "timers.h"
#include "RtosTask.h"

#include "eoss3_hal_gpio.h"
#include "dbg_uart.h"

extern const struct cli_cmd_entry my_main_menu[];

const char *SOFTWARE_VERSION_STR;

extern void qf_hardwareSetup();
static void nvic_init(void);

#define GPIO_OUTPUT_MODE (1)

void PyHal_GPIO_SetDir(uint8_t gpionum, uint8_t iomode);
int PyHal_GPIO_Set(uint8_t gpionum, uint8_t gpioval);

int main(void) {
   SOFTWARE_VERSION_STR = "qorc-onion-apps/qf_hello-fpga-gpio-ctlr";

   qf_hardwareSetup();
```

```
    nvic_init();

    dbg_str("\n\n");
    dbg_str("############################\n");
    dbg_str("Truth Table LED Demo\n");
    dbg_str("SW Version: ");
    dbg_str(SOFTWARE_VERSION_STR);
    dbg_str("\n");
    dbg_str(__DATE__ " " __TIME__ "\n");
    dbg_str("############################\n\n");

    CLI_start_task(my_main_menu);
    HAL_Delay_Init();

    // LED pins Output
    PyHal_GPIO_SetDir(18, GPIO_OUTPUT_MODE); // Blue LED
    PyHal_GPIO_SetDir(21, GPIO_OUTPUT_MODE); // Green LED
    PyHal_GPIO_SetDir(22, GPIO_OUTPUT_MODE); // Red LED

    // Truth table entries (minterms)
    int truth_table[] = {0, 1, 3, 11, 14};
    int num_entries = sizeof(truth_table) / sizeof(truth_table[0]);

    while (1) {
        for (int i = 0; i < num_entries; i++) {
            // Extract variables X, Y, Z, W from the truth table entry
            int X = (truth_table[i] >> 3) & 0x1;
            int Y = (truth_table[i] >> 2) & 0x1;
            int Z = (truth_table[i] >> 1) & 0x1;
            int W = truth_table[i] & 0x1;

            // Set LEDs based on the truth table entry
            PyHal_GPIO_Set(18, X);
            PyHal_GPIO_Set(21, Y);
            PyHal_GPIO_Set(22, Z || W); // For W or W', set the LED to 1

            HAL_DelayUSec(2000000); // Wait for 2 seconds
        }
    }

    // This part will never be reached
    vTaskStartScheduler();
    dbg_str("\n");
    while (1);
}

static void nvic_init(void) {
    // Set interrupt priorities
}

void SystemInit(void) {
    // System initialization
}

// Function to set GPIO direction
void PyHal_GPIO_SetDir(uint8_t gpionum, uint8_t iomode) {
    // Set GPIO direction
}

// Function to set GPIO value
int PyHal_GPIO_Set(uint8_t gpionum, uint8_t gpioval) {
    // Set GPIO value
}
```

Output:

X	Y	Z	W	$F(X,Y,Z,W)$
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

5.3.3 FPGA

Helloworldfpga.v code:

```
module helloworldfpga(
   output reg LCD_RS,
   output reg LCD_E,
   output reg[7:4] DATA
);

   wire clk;
   qlal4s3b_cell_macro u_qlal4s3b_cell_macro (
      .Sys_Clk0 (clk),
   ); // 20MHz clock

   integer i = 1;
   reg [25:0] count = 0;
   reg [3:0] Datas [1:41];

   // Solution text for "f = 1000 Hz"
   reg [7:0] solution[1:11] = {8'h66, 8'h20, 8'h3D, 8'h20, 8'h31, 8'h30, 8'h30, 8'h30, 8'h20, 8'h48, 8'h7A};

   initial begin
      // Initialize data for LCD
      // (This part remains unchanged from the original code)
   end

   always @(posedge clk) begin
      // Load solution text after clearing display
      if (i == 11) begin
```

```
        LCD_RS <= 1'b1; // Set RS for data
        DATA <= solution[i - 10]; // Load solution data
        LCD_E <= 1'b1;
        if (count == 800) begin // Wait 40us
            LCD_E <= 1'b0;
            count <= 0;
            i <= i + 1;
        end else
            count <= count + 1;
    end else if (i > 11 && i <= 20) begin // Continue loading solution text
        LCD_RS <= 1'b1; // Set RS for data
        DATA <= solution[i - 10]; // Load solution data
        LCD_E <= 1'b1;
        if (count == 800) begin // Wait 40us
            LCD_E <= 1'b0;
            count <= 0;
            i <= i + 1;
        end else
            count <= count + 1;
    end else if (i == 21) begin // Wait for 3ms after displaying solution
        if (count == 60000) begin // Wait 3ms
            count <= 0;
            i <= i + 1;
        end else
            count <= count + 1;
    end else if (i == 22) begin // Clear display after showing solution
        LCD_RS <= 1'b0; // Set RS for instruction
        DATA <= 4'h1; // Clear display instruction
        LCD_E <= 1'b1;
        if (count == 800) begin // Wait 40us
            LCD_E <= 1'b0;
            count <= 0;
            i <= i + 1;
        end else
            count <= count + 1;
    end else if (i > 22 && i <= 41) begin // Continue clearing display
        LCD_RS <= 1'b0; // Set RS for instruction
        DATA <= 4'h1; // Clear display instruction
        LCD_E <= 1'b1;
        if (count == 800) begin // Wait 40us
            LCD_E <= 1'b0;
            count <= 0;
            i <= i + 1;
        end else
            count <= count + 1;
    end else if (i > 41) begin // Reset loop counter
        i <= 1;
    end else begin // Load remaining initialization data for LCD
        // (This part remains unchanged from the original code)
    end
  end

endmodule
```

quickfeather.pcf:

```
# QuickFeather FPGA Pin Constraint File

# Clock signal
set_io -nowarn CLK pin_A4
```

```
# LCD RS, E, and DATA pins
set_io -nowarn LCD_RS pin_B2
set_io -nowarn LCD_E pin_B1
set_io -nowarn DATA[7] pin_C2
set_io -nowarn DATA[6] pin_C1
set_io -nowarn DATA[5] pin_C4
set_io -nowarn DATA[4] pin_C3
```

Output:

CONCLUSION

In conclusion, my internship experience at FWC, IIT Hyderabad in advanced 5G/6G wireless communications has been profoundly enriching and rewarding. Throughout this period, I have had the opportunity to delve into cutting-edge research and development in the field, gaining invaluable insights into the future of wireless technology.

Working alongside experts and fellow researchers, I have not only expanded my theoretical knowledge but also honed practical skills essential for tackling real-world challenges in the realm of wireless communications. From understanding the fundamentals of 5G and exploring its applications to delving into the nascent developments of 6G, this internship has provided me with a comprehensive understanding of the evolving landscape of telecommunications.

Moreover, the hands-on experience gained through various projects and experiments has equipped me with the requisite proficiency to navigate complex technical issues and innovate novel solutions. The collaborative environment fostered at FWC has encouraged brainstorming and knowledge sharing, amplifying the learning experience manifold.

As I reflect on the journey undertaken during this internship, I am confident that the insights gained and skills acquired will serve as a solid foundation for my future endeavors in the field of advanced wireless communications. I am immensely grateful for the guidance and mentorship received from the faculty and researchers at FWC, which have been instrumental in shaping my professional growth.

In essence, my internship at FWC, IIT Hyderabad has been an invaluable chapter in my academic and professional journey, propelling me towards a future where I aspire to contribute meaningfully to the advancements and innovations in wireless communications.